System Verification and Validation Plan for Keystone

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

November 3, 2026

Revision History

Date	Version	Notes
2026-10-29	1.0	General Information
2026-10-31	1.1	Plan and the SRS validation walkthrough appendix
2026-11-01	1.2	NFR System tests and referenced appendix parts
2026-11-01	1.3	FR System tests
2026-11-02	1.4	Requirement traceability tables

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 3

3 Plan **3**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 4
- 3.3 Design Verification . . . 4
- 3.4 Verification and Validation Plan Verification . . . 6
- 3.5 Implementation Verification . . . 6
- 3.6 Automated Testing and Verification Tools . . . 7
- 3.7 Software Validation . . . 8

4 System Tests **9**
- 4.1 Tests for Functional Requirements . . . 9
  - 4.1.1 Project Proposal Submission and Validation . . . 9
  - 4.1.2 Instructor Project Proposal . . . 11
  - 4.1.3 Project Discovery and Availability . . . 13
  - 4.1.4 Student Profiles and Team Formation . . . 15
  - 4.1.5 Project Recommendations, Applications, and Supervisor-Team Agreements . . . 21
  - 4.1.6 Project Communication and Notifications . . . 26
  - 4.1.7 Team-Project Approval and Validation . . . 31
  - 4.1.8 Course Configuration, Reference Materials, and Administration . . . 38
  - 4.1.9 Enrollment and Team Administration . . . 44
  - 4.1.10 Authentication, Authorization, and System Integrity . . . 48
- 4.2 Tests for Nonfunctional Requirements . . . 51
  - 4.2.1 Usability . . . 51
  - 4.2.2 Installability . . . 52
  - 4.2.3 Portability . . . 53
  - 4.2.4 Accessibility . . . 54
  - 4.2.5 Performance . . . 54
  - 4.2.6 Reliability . . . 55
  - 4.2.7 Maintainability . . . 56
  - 4.2.8 Security . . . 57
- 4.3 Traceability Between Test Cases and Requirements . . . 61

5 Unit Test Description **63**
- 5.1 Unit Testing Scope . . . 63
- 5.2 Tests for Functional Requirements . . . 63
  - 5.2.1 Module 1 . . . 63
  - 5.2.2 Module 2 . . . 64
- 5.3 Tests for Nonfunctional Requirements . . . 64

5.3.1 Module ? . . . 64
5.3.2 Module ? . . . 64
5.4 Traceability Between Test Cases and Modules . . . 65

6 Appendix **66**
6.1 Usability Test Proposal Values . . . 66
6.2 Usability Survey Questions . . . 66
6.3 SRS Validation Walkthrough and Checklist . . . 67
6.3.1 Agenda . . . 67
6.3.2 Questions for each use case . . . 67
6.3.3 Checklist . . . 68
6.4 Deployment Guide . . . 70
6.5 Load Test Task Mix . . . 71
6.6 Likely Change Sample . . . 71

7 Appendix — Generative AI Use Disclosure **72**
7.1 AI Tools Used . . . 72
7.2 How and Where Used . . . 72
7.3 Verification of AI-Generated Content . . . 73

8 Appendix — Reflection **74**

List of Tables

1 Verification and validation roles . . . 5
2 Automated testing and verification tools . . . 8
3 Traceability Between Functional Requirements and System Tests . . . 62
4 Traceability Between Nonfunctional Requirements and System Tests . . . 63
5 Proposal values for the supervisor task . . . 66
6 Agenda of the SRS walkthrough with Dr. Smith . . . 67
7 SRS validation checklist . . . 68
8 Role split and task probabilities for the load test . . . 71
9 Sample of likely changes for one week of maintenance . . . 72

List of Figures

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

This document describes how Keystone will be verified (are we building the product right?) and validated (are we building the right product?). Section 2 gives general information, including what is tested, what is not, and which documents the plan relies on. Section 3 describes the plan: who verifies what, how the SRS, design, this plan, and the implementation are verified, which tools are used, and how the software is validated with stakeholders. Section 4 describes the system tests, and Section 5 describes the unit tests. The appendix contains the material used to validate the SRS with the course instructors.

2 General Information

2.1 Summary

The software under test is Keystone, a web-based capstone project management system for the Software Engineering (SE), Computer Science (CS), and Mechatronics (TRON) capstone courses. Keystone covers the lifecycle from project proposal to team project approval. Its main functions are:

- structured project proposals by supervisors and student teams, with instructor review before a project is listed;
- a searchable project catalog showing project status and supervisor capacity;
- student and team profiles, team formation under course-specific rules, and teammate and project recommendations;
- team applications to projects, supervisor and team acceptance, and comment threads attached to projects and applications;
- team project submission with automated checks, and approval by instructors and TAs, either in the platform or recorded from an external process;
- course enrollment import from Avenue to Learn or file upload, validated team composition export, and email notifications and checklists;
- role-based access control, authentication through McMaster SSO or another trusted provider, and audit logging.

The requirements being verified are listed in the SRS (Patel et al., 2026f).

2.2 Objectives

The objectives are ordered by priority, based on the risks identified in the SRS: errors in the approval workflow affect course records, errors in access control expose student data, and a hard-to-use platform will not be adopted by supervisors.

In scope.

1. **Functional correctness.** Build confidence that the functional requirements (`FR_SupervisorProposal` to `FR_AssignTA` in the SRS) are met, especially the project and team-project status lifecycle, supervisor capacity (slot reservation and release), team formation rules, submission validation, enrollment import, and external supervisor verification and scoping.

2. **Access control and privacy.** Demonstrate that users can only see and change what their role and course permit, and that data is protected in transit and at rest (`NFR_Authenticate`, `NFR_Authorize`, `NFR_EncryptTransit`, `NFR_EncryptRest`, `DPR_RoleBasedDataPermissions`, `DPR_VisibilityMatrix`, `DPR_EncryptionAtRest`).

3. **Usability.** Demonstrate that first-time users in the main roles (students, TAs, and professors) can complete their primary tasks within the usability time limit, and find the platform intuitive and easy to use (`NFR_UsabilityFirstSuccess`, `NFR_LearnUnassisted`).

4. **Performance and installability.** Demonstrate that the system meets its latency target under the specified load, and that an IT professional can deploy it from the deployment guide within the time limit (`NFR_PerfTail`, `NFR_PeakUsers`, `NFR_DeployUnassisted`).

5. **Portability and accessibility.** Demonstrate that the system works on the target browsers and in Docker, and passes an automated accessibility audit (`NFR_Browsers`, `NFR_Container`, `NFR_AccessibilityWCAG`).

Out of scope.

- **Quality of recommendations.** We will test that recommendations are produced and displayed (`FR_TeamRecommend`), but not whether they are good matches. Recommendation quality is a stretch goal in the SRS, and judging it would need a study with real students.
- **External systems.** We assume that McMaster SSO, Avenue to Learn, the email service, and third-party libraries have been verified by their developers. We test our handling of their failures (for example, an unavailable Avenue), not their correct operation. Following the SRS Development Plan, email delivery and authentication may be simulated during the course.
- **Availability over time.** The availability criterion is measured over a long window (see the SRS), which cannot be observed within the course timeline. We will verify that the health check works and that the system meets the target over the period that the demo allows.
- **Maintainability.** We will review the likely changes in the SRS against the design, but will not measure the actual effort of making changes.
- **Features not implemented.** Requirements that the SRS Development Plan marks as future work, or as Phase 3 and not completed, will not be tested.
- **Formal verification.** We do not plan formal proofs.
- **Industry sponsors.** The usability test does not include external supervisors from industry, who have no McMaster account and are the least familiar with the course. Professors in the test represent internal supervisors and instructors. Findings about external supervisors will need follow-up testing if one becomes available.

2.3 Custom Documents

This project has two Custom Documents: a usability testing report and an interview report.

The **usability testing report** records the results of the usability test described in Section 4.2.1 and supports the Usability NFR in the SRS. In the test, we observe TAs, students, and professors using Keystone to complete the primary tasks of their roles, to judge whether the platform is

intuitive and easy to use without training. The report covers the participants and their roles, the tasks given, completion times, problems and points of confusion observed, and the changes made as a result.

The **interview report** records the stakeholder interviews that the requirements were derived from: students, external and internal supervisors, and the SE, TRON, and CS course instructors. It contains the interview notes, the requirements and design decisions each note led to, and the stakeholder feedback on the SRS and on the working system. It supports validation by checking that Keystone solves the problems stakeholders described, and the SRS (Appendix A) summarizes it.

2.4 Relevant Documentation

The following documents define what is verified and how the verification is organized.

- **Software Requirements Specification (SRS)** (Patel et al., 2026f). This is the main source for this plan. System tests are derived from its functional requirements, nonfunctional requirements and fit criteria, use cases, and data requirements. The SRS defines the symbolic constants (such as the number of concurrent users and the latency limit) that the nonfunctional tests use, and its traceability matrices will be extended to link each requirement to the tests in this document.
- **Hazard Analysis** (Patel et al., 2026b). It identifies hazards such as failures of the external approval system and access control failures, and the safety and security requirements that follow from them. Tests are added for each of these requirements.
- **Problem Statement and Goals** (Patel et al., 2026e). It records the problems and goals that validation checks against. Validation with stakeholders asks whether the system solves these problems, not only whether it meets the SRS.
- **Development Plan** (Patel et al., 2026a). It defines the tools and workflow (such as version control, continuous integration, and linting) that the automated testing in this plan relies on.
- **Module Guide (MG) and Module Interface Specification (MIS)** (Patel et al., 2026c,d). These will define the modules and interfaces that the unit tests in this plan target. The unit test sections will be completed after the design is finished.

3 Plan

This section describes who verifies and validates Keystone, and how. Section 3.1 assigns roles. Sections 3.2 to 3.5 describe the verification of the SRS, the design, this plan, and the implementation. Section 3.6 lists the automated tools, and Section 3.7 describes how the software is validated with stakeholders.

Handling review feedback. All reviews in this plan follow one process, which is not repeated in each subsection. Every finding from a review, walkthrough, inspection, or stakeholder meeting is recorded as a GitHub issue with the *Verification* label (and the *Documentation* or *Bug* label where relevant), and is assigned to a team member. The assignee fixes the finding on a branch and opens a pull request that references the issue. The pull request goes through the normal pull request review

described in the Development Plan (Patel et al., 2026a), and the issue is closed only after the pull request is merged and the reviewer who raised the finding, or a teammate, confirms that the change addresses it. A finding that the team decides not to act on is closed with a written reason.

3.1 Verification and Validation Team

Verification tasks rotate between team members, and no one verifies work that they wrote alone. The roles are in Table 1. The documentation and technical QA role rotates per deliverable in the order set in the Development Plan (Patel et al., 2026a): Ahmed, Andrew, Ayush, Waqar, and then repeats.

3.2 SRS Verification

The SRS (Patel et al., 2026f) is verified in four steps before it is treated as the basis for design and testing.

1. **Internal review.** Each team member reviews the sections that they did not write, using the SRS checklist from the course. The reviewer also checks the qualities that the SRS template asks for: unambiguous, consistent, complete, validatable, and abstract. The documentation and technical QA member then checks traceability: every stakeholder requirement maps to a use case and a functional requirement, every functional requirement maps to a stakeholder requirement or a nonfunctional requirement, every "Traces To" entry in the Hazard Analysis (Patel et al., 2026b) names a requirement that exists in the SRS, and every requirement has a fit criterion that can be tested.

2. **TA review.** Chris Schankula reviews the SRS and provides feedback.

3. **Peer review.** Classmates review the SRS using the Peer Review issue template, with the SRS checklist and the rubric.

4. **Validation with the instructors.** Reviewing the SRS ourselves cannot show that it matches how the courses really work, so Dr. Smith validates the facts that only he can confirm, through a structured walkthrough and a checklist. These are described below.

Walkthrough and checklist with Dr. Smith. We hold one 45 minute session with Dr. Smith, in the first supervisor meeting after the SRS is submitted and before the design document is started. The team presents instead of asking him to read the SRS, because the SRS is long and the questions that matter are about the use cases and the lifecycle. The agenda is in Table 6. We walk through the project and team-project status lifecycle and four use cases: `UC_ReviewProposal`, `UC_ApproveTeamProject`, `UC_ImportEnrollment`, and `UC_FollowUpUnselected`. These are the ones where the instructor's own process and decisions matter most. For each one, the presenter reads the main success scenario and the alternative flows and asks the targeted questions listed in Appendix 6.3. Dr. Smith then marks a short checklist of the statements in the SRS that only he can confirm, each as correct, needs a change, or unsure. The agenda, the questions, and the checklist are in Appendix 6.3. His answers are recorded in the Supervisor Meeting issue, and every "needs a change" or "unsure" answer becomes a Verification issue as described above.

Role	Members	Description
Documentation and technical QA	Rotates per deliverable: Ahmed, Andrew, Ayush, Waqar	Checks each deliverable against the repository checklist and the Avenue rubric before submission, runs the traceability check between documents, and flags gaps in time to fix them. The same member performs the post-merge functional QA for code in that deliverable period, and owns the system test results for it.
Unit test owner	Ayush Patel	Owns the backend (pytest) and frontend (Jest) unit tests and their coverage, and reviews unit tests in pull requests for whether they test the intended behaviour.
Integration test owner	Ahmed Rashrash	Owns the integration tests, including backend with the PostgreSQL database, and the interactions with the email service, Avenue to Learn, and the authentication providers.
End-to-end test owner	Andrew Habib	Owns the automated end-to-end (Playwright) tests for the most common use cases, including the cross-browser, screen-size, and accessibility checks.
CI/CD and automation owner	Waqar Ul-Hassan	Owns the GitHub Actions pipeline, the static analysis, coverage, and SonarQube setup, the performance test harness, and the mutation testing run, so that all checks run automatically as described in Section 3.6.
Verification issue tracking	Andrew Habib (GitHub Manager)	Keeps the *Verification* label, issue templates, and project board consistent, and checks before each milestone that no review finding is left open or unassigned.
Reviewers	All four team members	Review each other's pull requests and take part in the design and code walkthroughs.
Standing reviewer	Chris Schankula (TA)	Reviews and provides feedback on each deliverable
External reviewers and validators	Dr. Spencer Smith (supervisor and SE instructor), Dr. Mehdi Moradi (CS instructor), Dr. Alan Wassyng (TRON instructor).	Not members of the team. They judge whether the requirements and the software match how their courses work.

Table 1: Verification and validation roles

3.3 Design Verification

The design documents are the Module Guide and the Module Interface Specification (Patel et al., 2026c,d). They are verified as follows.

1. **Checklist review.** Teammates, the TA, and classmates review the documents using the course's MG and MIS checklists and the rubric.

2. **Requirements coverage check.** The documentation and technical QA member builds a table mapping every functional requirement and every safety and security requirement to the module(s) that satisfy it, and every module to the requirement(s) it exists for.

3. **Hazard scenario review.** For each hazard in the FMEA of the Hazard Analysis (Patel et al., 2026b), the team identifies the design element that mitigates it and checks that the mitigation is in the design and not only in the requirements. The review gives extra attention to the places where the design is most likely to fail such as the status lifecycle and slot reservation logic, simultaneous requests (race conditions), and the enforcement of visibility rules on every read path.

4. **Design walkthrough with Dr. Smith.** We walk through the module decomposition with him using scenarios, because modularity is his main design concern. Example scenarios are: an instructor disables a feature, a course uses external instead of in-platform approval, a course changes its team size rules after teams have formed, and a second university uses Keystone with a different enrollment source. For each scenario, we trace which modules change and which do not, and he judges whether the result is acceptable.

3.4 Verification and Validation Plan Verification

This plan is verified in the following ways.

1. **Review.** Teammates, the TA, and classmates review the plan using the course's VnV plan checklist and the rubric.

2. **Coverage check.** We confirm that every in-scope requirement in the SRS has at least one test, using the traceability table in this document. Requirements that carry the most risk (the status lifecycle and slot reservation, team size limits, role-based visibility, and enrollment import) must have more than one test, so that a passing test suite gives real confidence.

3. **Hazard check.** For each hazard in the Hazard Analysis, we ask which test would fail if the hazard occurred. A hazard with no such test is a finding.

4. **Validation review with Dr. Smith.** He reviews only the software validation plan (Section 3.7) and the usability test protocol, because this is where his input is most useful.

5. **Mutation testing (planned extra, high-risk areas only).** To check that the automated tests can detect faults, the team will run mutation testing on the highest-risk backend modules only. This includes the status lifecycle and slot reservation logic, the team formation rule checks, and the role-based visibility checks. The tool is mutmut. The run happens before the Revision 0 demo and before the final demo. Surviving mutants are examined, and each one that shows a missing test becomes a Verification issue. If time runs short, mutation testing is dropped without affecting any other part of this plan.

3.5 Implementation Verification

Implementation verification combines dynamic testing, static analysis, and a code walkthrough.

Dynamic testing

- **Unit tests (owner: Ayush).** Backend unit tests with pytest cover the API endpoints and the core logic. Frontend unit tests with Jest cover components, including form validation and the display of statuses. Test design follows the guidance in the Development Plan (Patel et al., 2026a), including boundary values and expected failures. The target is about 80% statement coverage. Section 5 describes the unit tests in detail.

- **Integration tests (owner: Ahmed).** These run the backend against a real PostgreSQL database in Docker, to test the transactions, constraints, and locking that the safety requirements depend on. The email service is replaced by a local test mail server (for example Mailpit), so tests can check what is sent, to whom, and how often. Avenue to Learn is replaced by a stub that can return a valid list, a malformed list, a duplicate entry, or no response, so that we test our handling of its failures, as the General Information section describes. Authentication providers are simulated in the same way.

- **End-to-end tests (owner: Andrew).** Playwright tests drive the real application in a browser through the most important use cases for each role. They run in Chrome, Firefox, and Safari (WebKit) at the three screen widths named in the SRS, and include an automated accessibility audit (axe-core) of every page in Chrome.

- **Performance tests (owner: Waqar).** A load test with Locust simulates the number of concurrent users, think time, and 95th percentile latency limit set in the nonfunctional requirement `NFR_PerfTail` of the SRS. The load test runs before milestones, not on every pull request, because it is slow and needs a full deployment. A first baseline run is done once the project catalog and application flow exist, and a run is done before each milestone from the Revision 0 demo onward, so that slow queries are found early.

Static verification

- **Static analyzers and linters.** Ruff and ESLint run on all code, as in the Development Plan (Patel et al., 2026a). We add a static type checker for the backend (mypy) and strict mode in TypeScript, because type errors in status and role handling are a likely source of faults. SonarQube analyzes code quality and coverage reports. Dependency audits (`pip-audit` and `npm audit`) check for known vulnerabilities.

- **Code walkthrough.** Before the Revision 0 demo, the team and the TA hold a walkthrough of the modules where a fault would cause the most harm: the status lifecycle and slot reservation logic, the visibility and role checks (including the external supervisor limits), and the notification queue and enrollment import. The author of a module presents it, and the other participants inspect it against a short checklist: the code follows the module interface specification, multi-step changes happen in one transaction, every read path checks permissions on the server, errors are handled explicitly, course rules are read from configuration and not written in the code, and each branch has a test. The session is recorded in a Team Meeting issue, and findings follow the process above. This is in addition to normal pull request review.

3.6 Automated Testing and Verification Tools

The team follows continuous integration and delivery (CI/CD) practice, using GitHub Actions as described in the Development Plan (Patel et al., 2026a). Table 2 lists the tools, what they are used for, and when they run. Pull requests cannot be merged while a required check fails. Falling below the 80% statement coverage target does not block a merge but is flagged for review, as the Development Plan states.

Tool	**Purpose**	**When it runs**
Ruff, ESLint	Linting and style for Python and TypeScript	Every push and pull request
mypy, TypeScript strict mode	Static type checking	Every push and pull request
pytest, pytest-cov	Backend unit tests and coverage (Cobertura XML)	Every push and pull request
Jest	Frontend unit tests and coverage (LCOV)	Every push and pull request
SonarQube	Code quality analysis, and coverage reporting from the pytest-cov and Jest reports	Every pull request and merge to main
pip-audit, npm audit	Known vulnerabilities in dependencies	Every pull request and merge to main
Docker with PostgreSQL, test mail server (Mailpit), Avenue stub	Integration and concurrency tests	Every pull request and merge to main
Playwright, axe-core	End-to-end tests in Chrome, Firefox, and WebKit, and the accessibility audit	Every pull request and merge to main
Locust	Load test at the load given in `NFR_PerfTail`	Manually started workflow before milestones (not on every pull request)
mutmut (planned extra)	Mutation testing of the high-risk backend modules	Manually started before the Revision 0 and final demos

Table 2: Automated testing and verification tools

3.7 Software Validation

Validation checks that we are building the right product: one that solves the problems in the Problem Statement (Patel et al., 2026e), and not only one that meets the SRS. It is done with stakeholders in the following ways. The details of each will be given later, in the usability test protocol (Section 4.2.1) and the Usability Testing Report.

- **Usability testing.** Students, TAs, and professors complete the primary tasks of their roles, as described in the Custom Documents section.
- **Stakeholder interviews.** We interview students, supervisors, and instructors again after the requirements and the first working version are available, to check that the platform addresses what they told us.
- **Demonstrations.** The Proof of Concept demo, the Revision 0 demo, and the final demo are used as validation points with Dr. Smith. After the Revision 0 demo, we also demonstrate the platform to Dr. Moradi and Dr. Wassyng and students.

As the General Information section states, external supervisors from industry are not part of the usability test.

4 System Tests

4.1 Tests for Functional Requirements

The tests below verify the functional requirements defined in Section 6 of the SRS. Tests are grouped by major functionalities including project proposals, team formation, project applications, approvals, communication, and course administration. Each test evaluates whether the system produces the expected results under the given conditions

4.1.1 Project Proposal Submission and Validation

This section covers the tests for project proposal submission and validation, as specified in FR_SupervisorPropos FR_ProposalOptionalFields, and FR_StudentProposal in Section 6 of the SRS. The tests verify successful supervisor and student proposal submissions, as well as the rejection of incomplete submissions and acceptance of proposals without optional information.

1. **T-FR-01: Submit a Complete Supervisor Proposal**

   Type: Automatic

   Initial State: A supervisor is authenticated, has permission to submit proposals, and the proposal submission period is open.

   Input: The supervisor completes and submits a proposal with the following information:

   - Title: Autonomous Waste Sorting System
   - Background: Manual waste sorting is inefficient and time-consuming.
   - Objectives: Develop a system that identifies and sorts recyclable materials.

- Required Skills: Python, computer vision, and mechanical design.
- Deliverables: Working prototype and technical documentation.
- Project Impact: Improved recycling efficiency.
- Supervisor Capacity: 2 teams
- Current Completion: Concept stage
- Project Type: Hardware
- Industry: Environmental technology
- Confidentiality/IP Concerns: None
- Additional Terms: No additional restrictions

Output: The system accepts the submission and stores the proposal with the "Proposal submitted" status making it available for instructors to review.

Test Case Derivation: Derived from `FR_SupervisorProposal` of the SRS. A proposal containing valid information for every field counts as a valid complete submission. This establishes a normal workflow where incomplete and invalid submissions can be compared to. Upon acceptance of the input, the system is expected to accept it and make it available for the instructor to look through.

How test will be performed: Using Playwright, authenticate as a supervisor, populate the proposal form with the predefined values, and submit it. Verify the confirmation and SUBMITTED status. Then, authenticate as an instructor and confirm that the proposal is accessible with the correct information.

2. **T-FR-02: Reject Missing Required Fields**

Type: Automatic

Initial State: A supervisor is authenticated, has permission to submit proposals, and the proposal submission period is open.

Input: A completed proposal template similar to T-FR-01 with one or more of the following required fields missing: title, background, course, objectives, impact, supervisor capacity details, level of current completion, industry, and information regarding confidentiality and IP concern. The supervisor will submit. Repeat for each of the following fields.

Output: The system rejects the submission, identifies the missing mandatory field(s), and prevents the proposal from entering the Proposal submitted state. The instructor will not receive the proposal.

Test Case Derivation: Derived from `FR_ProposalOptionalFields` of the SRS. Leaving out one mandatory field at a time isolates each validation constraint and verifies that all required fields are independently enforced.

How test will be performed: Using Playwright, populate the form with a valid baseline proposal. Remove one mandatory field, attempt submission, and verify that validation prevents submission. Repeat for each mandatory field. The test passes if every incomplete proposal is rejected and the corresponding missing field is identified. Repeated with as many combinations of mandatory fields as necessary.

3. **T-FR-03: Accept Missing Optional Fields**

   Type: Automatic

   Initial State: A supervisor is authenticated, has permission to submit proposals, and the proposal submission period is open.

   Input: A completed proposal template similar to T-FR-01 with all of the following required fields: title, background, course, objectives, impact, supervisor capacity details, level of current completion, industry, and information regarding confidentiality and IP concern. All optional fields will be left empty. The supervisor will submit.

   Output: The system accepts the submission and stores the proposal with the "Proposal submitted" status making it available for instructors to review.

   Test Case Derivation: Derived from `FR_ProposalOptionalFields` of the SRS. A proposal containing only mandatory fields represents the minimum valid submission and verifies that optional information is not incorrectly enforced as mandatory.

   How test will be performed: Using Playwright, complete only the required proposal fields and submit. Verify that the proposal is accepted despite the missing optional information and is accessible to the instructor. The test passes if both conditions are satisfied.

4. **T-FR-04: Publish a Student-Proposed Project**

   Type: Automatic

   Initial State: A student is authenticated and belongs to a team, and the proposal submission period is open.

   Input: The student publishes a proposal containing:

   - Title: Smart Campus Navigation
   - Problem Statement: Students have difficulty locating classrooms and accessible routes across campus.
   - Proposed Solution/Methodology: Develop a web application providing campus navigation and accessible route suggestions.

   Output: The proposal is stored, associated with the submitting team, and displayed in the separate student-proposed project list with the correct information.

   Test Case Derivation: Derived from `FR_StudentProposal`. A student team providing all specified project information verifies successful proposal creation and publication in the correct list.

   How test will be performed: Use Playwright to authenticate as a student belonging to a team, complete the student proposal template, and publish it. Verify the stored information, team association, and visibility in the student-proposed project list using an account authorized to view it.

4.1.2 Instructor Project Proposal

This section covers FR_ReviewProposal, including the approval, rejection, and revision workflows defined in UC_ReviewProposal. It also accounts for the updated external supervisor verification rule.

1. **T-FR-05: Approve a Supervisor Proposal**

   Type: Automatic

   Initial State: An authenticated instructor has access to an unapproved proposal titled Autonomous Waste Sorting System submitted by an unverified external supervisor. The proposal status is "Proposal submitted".

   Input: The instructor reviews the proposal and approves it.

   Output: The project status changes to "Open", and the project becomes visible in the approved catalog to authorized users. The supervisor is notified and becomes verified.

   Test Case Derivation: Derived from `FR_ReviewProposal` and `FR_ExternalSupervisorVerification`. Approving an unverified external supervisor's proposal verifies both the approval transition and the associated supervisor verification rule.

   How Test Will Be Performed: Use Playwright to approve the proposal as an instructor. Verify the project status, authorized catalog visibility, and supervisor notification. Then sign in as the external supervisor and verify that the account is marked verified and can access permitted student profiles. The test passes if all conditions are satisfied.

2. **T-FR-06: Request Changes to a Proposal**

   Type: Automatic

   Initial State: An instructor has access to a proposal with the "Proposal submitted" status, submitted by an unverified external supervisor.

   Input: The instructor requests changes and provides the feedback and comments asking for clarification on the expected project deliverables and scope.

   Output: The proposal status becomes "Proposal changes requested". The feedback is stored and communicated to the supervisor. The project remains unpublished, and the supervisor remains unverified.

   Test Case Derivation: Derived from `FR_ReviewProposal` and `FR_ExternalSupervisorVerification`. A revision request verifies that feedback does not trigger approval or external supervisor verification.

   How Test Will Be Performed: Use Playwright to request changes and submit the predefined feedback. Verify the status, stored feedback, and notification. Confirm that the project is excluded from the approved catalog and that the supervisor remains unverified.

3. **T-FR-07: Reject a Supervisor Proposal**

   Type: Automatic

Initial State: An authenticated instructor has access to a proposal with the "Proposal submitted" status, submitted by an unverified external supervisor.

Input: Reject the proposal with the reason "The proposed scope does not meet capstone course expectations."

Output: The proposal status changes to "Proposal rejected". The rejection reason is preserved, the supervisor is notified, and the project remains excluded from the approved catalog. The supervisor remains unverified.

Test Case Derivation: Derived from `FR_ReviewProposal` and `FR_ExternalSupervisorVerification`. Rejection verifies that an unsuccessful review does not grant approval-related permissions.

How Test Will Be Performed: Use Playwright to reject the proposal. Verify the updated status, recorded reason, and notification. Confirm that the project remains unpublished and the supervisor remains unverified.

4. **T-FR-08: Revise and Resubmit a Proposal**

   Type: Automatic

   Initial State: A supervisor proposal has the status "Proposal changes requested" and contains instructor feedback asking for clearer project deliverables.

   Input: The supervisor changes the deliverables from "Working prototype" to "Working prototype, technical documentation, and final demonstration", then resubmits.

   Output: The revised information is stored, the proposal returns to "Proposal submitted", and the instructor can review the updated version. Previous feedback remains in the review history.

   Test Case Derivation: Derived from `FR_ReviewProposal` and `UC_ReviewProposal`. Resubmission verifies the transition back to instructor review without losing earlier feedback.

   How Test Will Be Performed: Use Playwright to sign in as the supervisor, modify the deliverables, and resubmit. Verify the new information and status. Sign in as the instructor and confirm the revised proposal and earlier feedback are accessible.

4.1.3 Project Discovery and Availability

This section covers `FR_ViewApprovedProjects` and `FR_ViewProjectStatus` of the SRS. Tests verify search and filtering, project visibility restrictions, and availability based on reserved supervisor capacity.

1. **T-FR-09: Search and Filter Approved Projects**

   Type: Automatic

   Initial State: An authenticated student is enrolled in the course. Three approved projects are visible to the student:

   - Project A: Autonomous Waste Sorting System, Environmental Technology, Open.
   - Project B: Healthcare Scheduling System, Healthcare, Open.
   - Project C: Automated Assembly Line, Manufacturing, Full.

Input: Apply the industry filter "Environmental Technology", availability filter "Open", and search keyword "Autonomous".

Output: Only Project A is returned. Projects B and C are excluded because they do not satisfy every criterion.

Test Case Derivation: Derived from `FR_ViewApprovedProjects`. Projects with overlapping and distinct attributes verify that keyword searching and multiple filters are applied together.

How Test Will Be Performed: Populate the test environment with the three projects. Use Playwright to apply the filters and keyword. Compare returned project identifiers with the expected set {A}. The test passes if exactly Project A is returned.

2. **T-FR-10: Exclude Unapproved or Unauthorized Projects**

Type: Automatic

Initial State: The system contains three projects:

- Project A: Autonomous Waste Sorting System, approved with status "Open", submitted by External Supervisor X.
- Project B: Drone Inspection Platform, with status "Proposal submitted", awaiting instructor approval.
- Project C: Automated Assembly Line, approved with status "Full", submitted by External Supervisor Y.

A student enrolled in the course, the course instructor, and External Supervisors X and Y have authenticated accounts. Each external supervisor is associated only with their own project.

Input: The student searches for Projects A, B, and C. The instructor attempts to access all three projects, including Project B through the proposal review interface. External Supervisor X searches for Project C, while External Supervisor Y accesses Project C.

Output: The student can view Projects A and C, including Project C's "Full" status, but cannot view Project B because it has not been approved. The instructor can access all three projects, including Project B through the proposal review interface. External Supervisor X cannot view Project C because it belongs to another external supervisor, while External Supervisor Y can access Project C because they proposed it.

Test Case Derivation: Derived from `FR_ViewApprovedProjects` and `FR_ExternalSupervisorScope` of the SRS. Using projects with different approval states and ownership verifies that catalog visibility is restricted by both instructor approval and the user's authorization.

How Test Will Be Performed: Configure the three projects with the specified statuses and ownership. Using Playwright, authenticate as the student and search for Projects A, B, and C. Verify that Projects A and C are displayed, including Project C's "Full" status, while Project B is excluded. Authenticate as the instructor and verify access to all three projects, including Project B through the proposal review interface. Authenticate as External Supervisor X and search for Project C, verifying that it is excluded from results. Finally, authenticate as External Supervisor Y and verify that Project C is accessible. The test passes if each account can access only the project records permitted by the SRS visibility rules.

3. **T-FR-11: Display Project Availability and Capacity**

   Type: Automatic

   Initial State: An authenticated student can access two approved projects:

   - Project A: Maximum capacity of 2 teams with 1 supervisor slot reserved.
   - Project B: Maximum capacity of 1 team with 1 supervisor slot reserved.

   Input: The student opens the project catalog and views both projects.

   Output: Project A displays "Open" with a maximum capacity of 2 teams, 1 reserved slot, and 1 remaining available slot. Project B displays "Full" with a maximum capacity of 1 team, 1 reserved slot, and 0 remaining available slots.

   Test Case Derivation: Derived from `FR_ViewProjectStatus`. Remaining capacity is calculated as $C_{remaining} = C_{max} - R$ where R is reserved slot and C_{max} is maximum capacity. Project A has $2 - 1 = 1$ available slot, while Project B has $1 - 1 = 0$. These cases verify the correct availability states.

   How Test Will Be Performed: Configure both projects with the specified capacities and reserved slots. Use Playwright to view their catalog entries and compare the displayed statuses and maximum capacities with the expected values.

4. **T-FR-12: Prevent Applications to Full Projects**

   Type: Automatic

   Initial State: An authenticated student belongs to a registered team. An approved project has a maximum capacity of one team and its only supervisor slot is reserved, giving it the "Full" status.

   Input: The student attempts to submit an application to the full project.

   Output: The system rejects the application, informs the student that the project is unavailable, and does not create an application record.

   Test Case Derivation: Derived from `FR_ViewProjectStatus` and `UC_SelectProject`. A project with zero remaining capacity verifies that availability restrictions are enforced by the application workflow rather than only displayed visually.

   How Test Will Be Performed: Use Playwright to navigate to the full project and attempt to apply through the normal user interface. Verify that the application is blocked and that no new application record is created. The test passes if the full project cannot receive a new application.

4.1.4 Student Profiles and Team Formation

This section covers `FR_StudentProfile`, `FR_StudentTeams`, `FR_ValidateTeamRules`, `FR_TeamDiscovery`, and `FR_TeamProfile` in Section 6 of the SRS. The tests verify that students can create and maintain profiles, form teams, manage invitations, discover available teammates, and comply with course-specific team formation rules. The tests also verify the handling of invalid team configurations and changes to existing rules.

1. **T-FR-13: Create and View a Student Profile**

   Type: Automatic

   Initial State: A student is authenticated and enrolled in the Software Engineering capstone course. The student has not created a profile.

   Input: The student creates a profile with:

   - Bio: Fourth-year Software Engineering student interested in developing AI applications.
   - Program: Software Engineering
   - Interests: Artificial intelligence and robotics
   - Project Preferences: AI and automation projects
   - Skills: Python, Java, React
   - Experience: Software development internship
   - Work Style: Collaborative and organized
   - LinkedIn: https://www.linkedin.com/in/teststudent

   The student saves the profile.

   Output: The system successfully stores the profile with all provided information. The student can reopen the profile and view the saved values. Other authorized students and supervisors can also view the profile.

   Test Case Derivation: Derived from `FR_StudentProfile`. Providing values for the specified profile attributes verifies that students can create persistent profiles containing information used for teammate and project discovery.

   How Test Will Be Performed: Using Playwright, authenticate as the student, complete the profile form, and save it. Reopen the profile and verify all stored fields. Authenticate as another authorized student and an approved supervisor to verify profile visibility.

2. **T-FR-14: Update an Existing Student Profile**

   Type: Automatic

   Initial State: An authenticated student has an existing profile with the skills "Python, Java, React" and project preferences "AI and automation projects" (Profile provided in T-FR-13).

   Input: The student updates the skills to "Python, Java, React, PyTorch" and changes the project preferences to "Machine learning and computer vision". The student saves the changes.

   Output: The updated profile contains the new skills and project preferences. Unmodified information remains unchanged, and the revised values are visible when the profile is reopened.

   Test Case Derivation: Derived from `FR_StudentProfile`. Changing selected profile fields verifies that students can maintain their information without unintentionally modifying or losing unrelated profile attributes.

   How Test Will Be Performed: Using Playwright, open the existing profile, change the specified fields, and save. Reload the profile and compare all stored fields with their expected values. Confirm that the updated information is visible to authorized users.

3. **T-FR-15: Create a Student Team and Assign Roles**

   Type: Automatic

   Initial State: Three authenticated students, Student A, Student B, and Student C, are enrolled in the Software Engineering capstone course and do not belong to any existing teams. The course permits teams of 3 to 5 students.

   Input: Student A creates a team named "Amazing Engineers", adds Student B and Student C through the supported team formation workflow, and assigns Student A as the team's point of contact.

   Output: The team is created with the three students as members. Student A is identified as the point of contact, and the team membership and assigned roles are stored correctly.

   Test Case Derivation: Derived from `FR_StudentTeams`. Creating a team with a valid number of members and assigning a point of contact verifies that students can organize themselves into teams and define responsibilities.

   How Test Will Be Performed: Configure the course team size limits and create the three student accounts. Using Playwright, create the team, complete the necessary membership invitations or confirmations, and assign the point-of-contact role. Verify the team membership and recorded role assignments.

4. **T-FR-16: Send and Accept a Team Invitation**

   Type: Automatic

   Initial State: Team Amazing Engineers has 3 members and a maximum permitted size of 5 as defined in T-FR-15. Student D is enrolled in the same course and does not belong to another team.

   Input: Student A invites Student D to join Team Amazing Engineers. Student D opens the invitation and accepts.

   Output: Student D becomes a member of Team Amazing Engineers. The membership count increases from 3 to 4, and Student D appears in the team's member list. The invitation is no longer pending.

   Test Case Derivation: Derived from `FR_StudentTeams`. Accepting an invitation verifies that team invitations can be received and that accepted invitations update the team's membership.

   How Test Will Be Performed: Using Playwright, authenticate as Student A and invite Student D. Authenticate as Student D, accept the invitation, and verify that both accounts display the updated team membership and invitation status.

5. **T-FR-17: Decline a Team Invitation**

   Type: Automatic

   Initial State: Team Amazing Engineers has 3 members and can recruit additional students. Student E is enrolled in the course, has no team, and has received an invitation to join Team Amazing Engineers.

Input: Student E declines the invitation.

Output: The invitation is recorded as declined. Student E is not added to Team Amazing Engineers, and the team's membership count remains 3.

Test Case Derivation: Derived from `FR_StudentTeams`. Declining an invitation verifies that students can reject team membership requests without changing existing team composition.

How Test Will Be Performed: Using Playwright, authenticate as Student A and invite Student E. Authenticate as Student E and decline the invitation. Verify that the invitation is no longer pending, Student E remains without a team, and Team Amazing Engineers's member list is unchanged.

6. **T-FR-18: Find and Join an Existing Team**

Type: Automatic

Initial State: Team Amazing Engineers has 3 members and a maximum permitted size of 5. The team is configured to recruit 2 additional members. Student F is enrolled in the course, is not on a team, and is eligible to join Team Amazing Engineers. Student A is an existing member of Team Amazing Engineers.

Input: Student F discovers Team Amazing Engineers through the team recruitment interface and submits a request to join. Student A reviews and accepts the request.

Output: Student F becomes a member of Team Amazing Engineers, increasing membership to 4. The team now requires only 1 additional member, and the membership information is updated for all current members.

Test Case Derivation: Derived from `FR_StudentTeams`. Joining through an open recruitment opportunity verifies that students can join existing teams without requiring a direct invitation from an existing team member. It should also verify that any student on the team can accept requests.

How Test Will Be Performed: Using Playwright, configure Team Amazing Engineers to recruit additional members. Authenticate as Student F, find the team, and submit a join request. Authenticate as Student A, review and accept Student F's request through the team interface, and verify the updated membership and recruitment count.

7. **T-FR-19: Validate Team Size Constraints**

Type: Automatic

Initial State: The Software Engineering capstone course requires a minimum of 3 students and a maximum of 5 students per team. Team Amazing Engineers contains 2 students, Team Fantastic Coders contains 3 students, and Team Capstone Legends contains 5 students. Team Amazing Engineers is recruiting additional members. Team Amazing Engineers has reached an agreement with the supervisor of Autonomous Waste Sorting System, and its team-project status is "Agreed". In-platform approval is enabled, and all other required submission information is complete.

Input: Team Amazing Engineers attempts to submit its team-project information for instructor approval with only 2 members. Team Fantastic Coders and Team Capstone Legends

are checked against the configured team-size requirements. Separately, a student who is not on a team attempts to join Team Capstone Legends, which already contains 5 members.

Output: Team Amazing Engineers is identified as not meeting the minimum team size and its submission for instructor approval is rejected with a validation message. Team Amazing Engineers remains intact and may continue recruiting additional members. Team Fantastic Coders and Team Capstone Legends satisfy the configured team-size requirements. The student's attempt to join Team Capstone Legends is rejected because the maximum team size would be exceeded, and Team Capstone Legends' membership remains unchanged at 5 students.

Test Case Derivation: Derived from `FR_ValidateTeamRules` and `FR_ApprovalPreflightValidate`. For minimum size $m = 3$ and maximum size $M = 5$, compliant team sizes satisfy $m \leq n \leq M$. Testing $n = 2, 3, 5, 6$ verifies the minimum-size restriction before instructor approval, valid boundary values, and maximum-size enforcement during team membership changes.

How Test Will Be Performed: Configure the course team-size limits and prepare Team Amazing Engineers, Team Fantastic Coders, and Team Capstone Legends with their specified membership counts. Using Playwright, authenticate as a member of Team Amazing Engineers and attempt to submit its team-project information for instructor approval. Verify that the submission is rejected, the minimum-size violation is identified, and the team-project status remains "Agreed". Verify that Team Fantastic Coders and Team Capstone Legends satisfy the configured team-size requirements. Authenticate as a student who is not on a team and attempt to join Team Capstone Legends. Verify that the join attempt is rejected and Team Capstone Legends retains its original 5 members.

8. **T-FR-20: Reject Disallowed Program Combinations**

Type: Automatic

Initial State: The course configuration permits teams consisting entirely of Software Engineering students or a combination of Software Engineering and Mechatronics students. Teams combining Software Engineering and Computer Science students are not permitted. The minimum team size is 3. Team A and Team B each initially contain 2 Software Engineering students.

Input:

- A Mechatronics student attempts to join Team A, increasing its membership to 3.
- A Computer Science student attempts to join Team B, which would increase its membership to 3.

Output: The Mechatronics student successfully joins Team A, resulting in a valid team of 3 members. The Computer Science student's attempt to join Team B is rejected because the program combination is not permitted. Team B retains its original 2 Software Engineering members, and a validation message identifies the violated program-combination rule.

Test Case Derivation: Derived from `FR_ValidateTeamRules`. Testing one permitted and one disallowed combination verifies that the system enforces configurable program restrictions rather than allowing every combination of enrolled students.

How Test Will Be Performed: Configure the permitted program combinations and prepare both teams using the specified student programs. Using Playwright, initiate team validation and verify that Team A passes while Team B is rejected.

9. **T-FR-21: Prevent Duplicate Team Membership**

   Type: Automatic

   Initial State: Student A is already a member of Team Amazing Engineers. Team Fantastic Coders is an existing team with an available membership slot. Both teams belong to the same capstone course.

   Input: Student A attempts to join Team Fantastic Coders while remaining a member of Team Amazing Engineers.

   Output: The system rejects Student A's attempt to join Team Fantastic Coders. Student A remains a member of Team Amazing Engineers only, and neither team's membership is incorrectly modified.

   Test Case Derivation: Derived from `FR_ValidateTeamRules`. Attempting to join a second team verifies that duplicate membership is prevented and that a student belongs to at most one team.

   How Test Will Be Performed: Configure the two teams and their membership records. Using Playwright, authenticate as Student A and attempt to join Team Fantastic Coders. Verify the rejection and confirm that Student A remains associated only with Team Amazing Engineers.

10. **T-FR-22: Flag Existing Teams After Rule Changes**

    Type: Automatic

    Initial State: A course permits team sizes from 3 to 5 students. Team Amazing Engineers has 3 members, and Team Fantastic Coders has 5 members. Both teams are valid under the existing rules.

    Input: The instructor changes the minimum team size from 3 to 4 while retaining the maximum team size of 5.

    Output: The system identifies Team Amazing Engineers as non-compliant with the proposed rules. After the change is applied, Team Amazing Engineers remains intact but is flagged for instructor review. Team Fantastic Coders remains valid and is not flagged.

    Test Case Derivation: Derived from `FR_ValidateTeamRules`. Changing the minimum size from $m = 3$ to $m = 4$ makes Team Amazing Engineers's membership $n = 3 < m$, while Team Fantastic Coders continues to satisfy $4 \leq n \leq 5$. This verifies that changes to formation rules identify newly non-compliant teams without automatically invalidating or dissolving them.

    How Test Will Be Performed: Configure the initial course rules and create both teams. Using Playwright, authenticate as an instructor and change the minimum team size. Verify that Team Amazing Engineers is identified as affected before confirmation. Apply the rule change and confirm that Team Amazing Engineers is flagged while both teams retain their original memberships.

11. **T-FR-23: Discover Students and Teams Seeking Members**

Type: Automatic

Initial State: Students A and B are enrolled in the same course and have not joined a team. Team Amazing Engineers is recruiting 2 additional members, while Team Fantastic Coders is not recruiting.

Input: Student A marks their profile as "Looking for a team". Student B leaves their availability disabled. Another authenticated student opens the teammate and team discovery interface.

Output: Student A appears among students seeking a team, while Student B does not. Team Amazing Engineers appears among teams recruiting additional members, while Team Fantastic Coders does not.

Test Case Derivation: Derived from `FR_TeamDiscovery`. Using students and teams with different recruitment states verifies that the discovery interface reflects which users and teams are actively seeking members.

How Test Will Be Performed: Using Playwright, configure the specified student availability and team recruitment statuses. Authenticate as another student and open the discovery interface. Verify that only Student A and Team Amazing Engineers appear in their respective discovery results.

12. **T-FR-24: Create, Update, and View a Team Profile**

Type: Automatic

Initial State: Team Amazing Engineers consists of three registered students. The team has not created a shared team profile.

Input: An authorized team member creates a profile containing:

- Team Bio: A team focused on practical software and robotics solutions.
- Collective Skills: Python, React, embedded systems
- Project Interests: Robotics and automation
- Goals: Develop a functional prototype
- Work Style: Weekly meetings and shared task management

The member saves the profile, then updates the collective skills to include "Computer vision".

Output: The shared team profile is stored with the specified information. After editing, the collective skills include computer vision, and all other profile fields remain unchanged. The profile is visible to authorized users.

Test Case Derivation: Derived from `FR_TeamProfile`. Creating, editing, and viewing a shared profile verifies that teams can maintain consolidated information about their members' skills, interests, goals, and work style for other authorized stakeholders.

How Test Will Be Performed: Using Playwright, authenticate as an authorized Team Amazing Engineers member, create the team profile, and save it. Reopen the profile, update the collective skills, and verify that the change persists. Authenticate as another authorized user and verify that the shared profile and updated information are visible.

4.1.5 Project Recommendations, Applications, and Supervisor-Team Agreements

This section covers `FR_TeamRecommend`, `FR_ProjectInterest`, and `FR_DeclineTeam` in Section 6 of the SRS. The tests verify that students receive relevant recommendations, submit applications to available projects, and record mutual agreements with supervisors. The tests also verify the capacity and status transitions defined in Section 7.2 of the SRS.

1. **T-FR-25: Recommend Compatible Projects and Teammates**

   Type: Automatic

   Initial State: Student A has a complete profile containing Python, machine learning, and computer vision skills, with interests in artificial intelligence and automation. Three available projects and two students seeking teammates exist:

   - Project A: Computer Vision Defect Detection, requiring Python and computer vision skills.
   - Project B: Mechanical Gearbox Design, requiring CAD and mechanical design skills.
   - Project C: AI-Based Quality Inspection, requiring Python and machine learning skills.
   - Student B: Interested in artificial intelligence with Python and machine learning experience.
   - Student C: Interested in mechanical design with CAD and SolidWorks experience.

   Input: Student A opens the project and teammate recommendation interface.

   Output: The system generates recommendations based on Student A's completed profile. Projects A and C are identified as stronger matches than Project B, and Student B is identified as a stronger teammate match than Student C, based on overlapping attributes.

   Test Case Derivation: Derived from `FR_TeamRecommend`. Providing candidates with strongly overlapping and distinct skills and interests verifies that recommendations account for profile similarity rather than presenting unrelated candidates without considering the student's preferences.

   How Test Will Be Performed: Create the three projects and two student profiles with the specified skills and interests. Using Playwright, log in as Student A and open the recommendations page. Verify that Projects A and C are recommended ahead of Project B, and that Student B is recommended ahead of Student C, based on matching skills and interests.

2. **T-FR-26: Submit an Application to an Open Project**

   Type: Automatic

   Initial State: Team Amazing Engineers consists of three registered students, has no existing application to Autonomous Waste Sorting System, and has not accepted another project. Autonomous Waste Sorting System has status "Open", maximum capacity $C_{\max} = 2$, and $R = 0$ reserved slots.

   Input: A member of Team Amazing Engineers opens Autonomous Waste Sorting System and submits an application expressing the team's interest.

Output: An application associated with Team Amazing Engineers and the project is created with the team-project status "Applied". The project supervisor can review the application. The project remains "Open" and $R = 0$, since an application does not reserve a supervisor slot.

Test Case Derivation: Derived from `FR_ProjectInterest` and `UC_SelectProject`. A valid application verifies that teams can express interest in an available supervisor-proposed project without immediately occupying its capacity.

How Test Will Be Performed: Using Playwright, authenticate as a member of Team Amazing Engineers, open the project, and submit the application. Verify the "Applied" status and team-project association. Authenticate as the supervisor and verify that the application is available for review. Confirm that the project remains "Open" with zero reserved slots.

3. **T-FR-27: Reserve Capacity When a Supervisor Accepts a Team**

   Type: Automatic

   Initial State: Autonomous Waste Sorting System has maximum capacity $C_{max} = 2$ and $R = 1$ reserved slot. Team Fantastic Coders has an application with status "Applied" and has not been accepted by the supervisor.

   Input: The supervisor accepts Team Fantastic Coders's application.

   Output: Team Fantastic Coders's status becomes "Supervisor Accepts", the supervisor's acceptance is recorded, and the number of reserved slots increases from 1 to 2. The project status changes from "Open" to "Full". Team Fantastic Coders has not yet reached "Agreed" because the team has not accepted.

   Test Case Derivation: Derived from `FR_DeclineTeam` and the capacity rules in Section 7.2 of the SRS. Remaining capacity is calculated as $C_{remaining} = C_{max} - R$. Initially, $C_{remaining} = 2 - 1 = 1$. After supervisor acceptance, $R \leftarrow R + 1 = 2$, resulting in $C_{remaining} = 2 - 2 = 0$. These values verify that supervisor acceptance reserves exactly one slot and updates the project to "Full".

   How Test Will Be Performed: Configure the project and Team Fantastic Coders's pending application. Using Playwright, authenticate as the supervisor and accept Team Fantastic Coders. Verify the updated team-project status, acceptance record, reserved slot count, and project availability.

4. **T-FR-28: Record Team Acceptance Before Supervisor Acceptance**

   Type: Automatic

   Initial State: Team Amazing Engineers has an application to Autonomous Waste Sorting System with status "Applied". The project has maximum capacity $C_{max} = 2$, $R = 0$ reserved slots, and status "Open".

   Input: Team Amazing Engineers accepts the proposed working arrangement before receiving supervisor acceptance.

   Output: The team-project status becomes "Team Accepts - Supervisor Pending". The team's acceptance is recorded, but the arrangement is not marked "Agreed". No supervisor slot is reserved, so $R = 0$ and the project remains "Open".

Test Case Derivation: Derived from `FR_DeclineTeam` and the status lifecycle in Section 7.2 of the SRS. Team acceptance alone must not reserve project capacity or establish a mutual agreement.

How Test Will Be Performed: Using Playwright, authenticate as an authorized Team Amazing Engineers member and record the team's acceptance. Verify the pending-supervisor status and confirm that the project capacity and availability remain unchanged.

5. **T-FR-29: Establish a Mutual Supervisor-Team Agreement**

   Type: Automatic

   Initial State: Team Amazing Engineers has applied to Autonomous Waste Sorting System. The supervisor has already accepted the team, reserving one slot. The team-project status is "Supervisor Accepts". The team has not yet accepted the arrangement.

   Input: An authorized Team Amazing Engineers member accepts the working arrangement.

   Output: The team-project status changes to "Agreed". Both the supervisor's and team's acceptance decisions are recorded. The reserved slot remains occupied without incrementing again.

   Test Case Derivation: Derived from `FR_DeclineTeam` and Section 7.2 of the SRS. Recording the second acceptance verifies that a mutual agreement is established only when both parties accept. Since the supervisor has already reserved the slot, the second acceptance must not increase R again.

   How Test Will Be Performed: Prepare the application with the supervisor's acceptance already recorded. Using Playwright, authenticate as Team Amazing Engineers and accept the arrangement. Verify the "Agreed" status, both acceptance records, and unchanged reserved-slot count.

6. **T-FR-30: Decline a Team Application**

   Type: Automatic

   Initial State: Team Fantastic Coders has submitted an application to an "Open" project. The application has status "Applied", and the supervisor has not accepted the team or reserved a slot.

   Input: The supervisor declines Team Fantastic Coders's application and provides the optional reason: "The project requires a different combination of technical skills."

   Output: The application changes to "Declined or withdrawn". The decision is communicated through a respectful message containing the supplied reason. No supervisor slot is reserved or released, and the project remains "Open".

   Test Case Derivation: Derived from `FR_DeclineTeam`. Declining an application before supervisor acceptance verifies that supervisors can reject applications, communicate the decision respectfully, and capacity remains unchanged.

   How Test Will Be Performed: Using Playwright, authenticate as the supervisor, decline Team Fantastic Coders's application, and enter the specified reason. Verify the updated status, recorded reason, notification, and unchanged project capacity. Confirm that the message uses the system's respectful decline template.

7. **T-FR-31: Release Reserved Capacity After Withdrawal**

   Type: Automatic

   Initial State: Autonomous Waste Sorting System has maximum capacity $C_{max} = 2$ and $R = 2$ reserved slots, giving it status "Full". Team Fantastic Coders occupies one reserved slot because its supervisor previously accepted its application.

   Input: Team Fantastic Coders withdraws its application through the supported withdrawal workflow.

   Output: Team Fantastic Coders's application changes to "Declined or withdrawn". Its reserved slot is released, reducing R from 2 to 1. The project becomes "Open" and is available for additional team applications.

   Test Case Derivation: Derived from `FR_DeclineTeam`, `UC_SelectProject`, and the capacity rules in Section 7.2 of the SRS. Initially, $C_{remaining} = C_{max} - R = 2 - 2 = 0$. Following withdrawal, $R \leftarrow R - 1 = 1$, resulting in $C_{remaining} = 2 - 1 = 1$. These transitions verify that releasing an occupied slot restores availability without changing maximum capacity.

   How Test Will Be Performed: Configure the project with two reserved slots and Team Fantastic Coders as one of the accepted teams. Using Playwright, authenticate as Team Fantastic Coders and withdraw its application. Verify the withdrawal status, released slot, and change from "Full" to "Open".

8. **T-FR-32: Prevent Supervisor Acceptance Beyond Capacity**

   Type: Automatic

   Initial State: Project Automated Assembly Line has maximum capacity $C_{max} = 1$ and $R = 1$ reserved slot, giving it status "Full". Team Amazing Engineers has a previously submitted application with status "Applied", created before the project became full.

   Input: The supervisor attempts to accept Team Amazing Engineers' application.

   Output: The acceptance is rejected because no capacity remains. Team Amazing Engineers does not acquire a reserved slot, the existing accepted team remains unchanged, and $R = 1$.

   Test Case Derivation: Derived from `FR_DeclineTeam` and Section 7.2 of the SRS. Supervisor acceptance must preserve $0 \leq R \leq C_{max}$. Attempting another acceptance when $R = C_{max} = 1$ would produce $R + 1 = 2 > C_{max}$, violating the capacity constraint. The system must therefore prevent the action.

   How Test Will Be Performed: Prepare the full project and an existing pending application. Using Playwright, authenticate as the supervisor and attempt acceptance. Verify that the action is blocked and that neither the reservation count nor the existing team assignments change.

9. **T-FR-33: Release Other Reservations After Team Agreement**

   Type: Automatic

   Initial State: Team Amazing Engineers has applications to two approved projects:

- Project A: Autonomous Waste Sorting System, capacity $C_{max} = 2$, with $R = 1$ reserved slot held for Team Amazing Engineers.
- Project B: Automated Assembly Line, capacity $C_{max} = 1$, with $R = 1$ reserved slot held for Team Amazing Engineers.

Both supervisors have accepted Team Amazing Engineers. The team has not yet accepted either project.

Input: Team Amazing Engineers accepts Project A's working arrangement.

Output: Team Amazing Engineers reaches "Agreed" on Project A, retaining its reservation there. Its application to Project B is withdrawn, releasing the reservation. Project B's reserved slots decrease from 1 to 0, and its status changes from "Full" to "Open".

Test Case Derivation: Derived from `FR_DeclineTeam` and Section 7.2 of the SRS. A team's agreement with one project must release reservations associated with competing applications. For Project B, $R \leftarrow R - 1 = 0$ and $C_{remaining} = 1 - 0 = 1$. This verifies that an agreement does not leave another project's capacity unnecessarily occupied.

How Test Will Be Performed: Configure both projects with Team Amazing Engineers's supervisor-accepted applications. Using Playwright, authenticate as Team Amazing Engineers and accept Project A. Verify the "Agreed" status on Project A, withdrawal of the Project B application, and restoration of Project B's availability.

10. **T-FR-34: Prevent Duplicate Supervisor Acceptance**

Type: Automatic

Initial State: A project has maximum capacity $C_{max} = 2$ and $R = 1$ reserved slot. Team Amazing Engineers's application has status "Supervisor Accepts", and the supervisor's acceptance has already been recorded.

Input: The supervisor attempts to accept Team Amazing Engineers's application a second time.

Output: The repeated action does not create another reservation nor increment the reserved-slot count. The team-project status remains "Supervisor Accepts" and $R = 1$.

Test Case Derivation: Derived from `FR_DeclineTeam` and the capacity rules in Section 7.2 of the SRS and the PoC. Each supervisor-accepted team must occupy exactly one slot. Repeated acceptance of the same application must not produce $R \leftarrow R + 1$ again.

How Test Will Be Performed: Configure an application with supervisor acceptance already recorded. Using Playwright, attempt to repeat the acceptance through the available interface or resend the same supported action. Verify that no additional reservation is created and that the stored status and capacity remain unchanged.

4.1.6 Project Communication and Notifications

This section covers `FR_ProjectThreads` and `FR_ThreadsNotify` in Section 6 of the SRS, along with the access restrictions defined in `FR_ExternalSupervisorScope`. The tests verify that authorized users can create and reply to discussion threads associated with project proposals, team formations, and project applications. They also verify that relevant users receive email notifications for thread

activity, application decisions, and project status changes, while ensuring that notification recipients are authorized and confidential information is not disclosed.

1. **T-FR-35: Create a Discussion Thread on a Project Proposal**

   Type: Automatic

   Initial State: The Autonomous Waste Sorting System proposal exists with status "Proposal submitted". Supervisor A is its owner and is authenticated.

   Input: Supervisor A creates a thread with the subject "Project Deliverable Requirements" and the message "Are student teams required to produce a working physical prototype, or can the final deliverable be a software simulation?"

   Output: A new thread with the specified subject and message is created and associated with the Autonomous Waste Sorting System proposal. The author and creation timestamp are recorded, and the thread is accessible to authorized users.

   Test Case Derivation: Derived from `FR_ProjectThreads`, which requires authorized users to create threads associated with their own project proposals. Creating a thread on a submitted proposal verifies the ability to initiate project-related communication.

   How Test Will Be Performed: Using Playwright, authenticate as Supervisor A, navigate to the proposal, and create the specified thread. Verify that the thread appears under the correct proposal and that its subject, content, author, and timestamp match the submitted information.

2. **T-FR-36: Reply to a Discussion Thread**

   Type: Automatic

   Initial State: The Autonomous Waste Sorting System proposal contains a thread titled "Project Deliverable Requirements" asking the question proposed in T-FR-35, created by Supervisor A. Instructor A is assigned to the course and is authorized to review the proposal. The thread currently contains no replies.

   Input: Instructor A replies: "Student teams may develop either a physical prototype or a software simulation, depending on the project's scope and learning objectives. The expected deliverables should be clearly specified in the proposal."

   Output: The reply is appended to the existing thread with Instructor A identified as its author and a timestamp recording when the reply was submitted. The original message, including its content, author, and timestamp, remains unchanged. Both messages are displayed in chronological order under the same thread.

   Test Case Derivation: Derived from `FR_ProjectThreads`. Allowing an authorized instructor to respond verifies that threads support ongoing communication, rather than only the creation of standalone messages.

   How Test Will Be Performed: Using Playwright, authenticate as Instructor A, open the existing thread, and submit the reply. Verify that both messages are visible, the reply is attributed to Instructor A, and the original message is preserved.

3. **T-FR-37: Communicate Through Team Formation and Application Threads**

   Type: Automatic

   Initial State: Team Amazing Engineers consists of three students, including Student A and Student B. Team Amazing Engineers has an application to Autonomous Waste Sorting System with status "Applied". No threads are associated with the team or application.

   Input:

   (a) Student A creates a team formation thread titled "Meeting Availability" with the message "Can we meet on Friday?"

   (b) Student B replies "Friday afternoon works."

   (c) Student A creates a thread on the project application titled "Project Requirements" with the message "Can we clarify the hardware requirements?"

   Output: Both threads are created under their corresponding records. The team formation thread contains Student A's message and Student B's reply. The project application thread contains the specified requirements question. Neither thread is associated with the wrong record.

   Test Case Derivation: Derived from `FR_ProjectThreads`, which supports threads on team formations and project applications in addition to proposals. Testing both record types verifies that discussion functionality is available throughout the team and project selection process.

   How Test Will Be Performed: Using Playwright, authenticate as Student A and create both threads. Authenticate as Student B to reply to the team formation thread. Verify the content, authors, replies, and correct record associations.

4. **T-FR-38: Notify Thread Participants of a New Reply**

   Type: Automatic

   Initial State: Supervisor A and Instructor A participate in the "Project Deliverable Requirements" thread associated with Autonomous Waste Sorting System. Supervisor A previously asked whether student teams must produce a physical prototype or whether a software simulation is sufficient. Both users have valid email addresses and remain authorized to access the thread.

   Input: Instructor A posts the reply: "Student teams may develop either a physical prototype or a software simulation, depending on the project's scope and learning objectives. The expected deliverables should be clearly specified in the proposal."

   Output: The reply is appended to the existing thread with Instructor A identified as its author and a submission timestamp. Supervisor A receives an email notification identifying the new reply and containing a link to the associated discussion. The email contains only an event summary and link, without disclosing confidential project information.

   Test Case Derivation: Derived from `FR_ThreadsNotify`, which requires email notifications when a new reply is posted in a thread a user participates in. The scenario verifies that Supervisor A receives a notification following Instructor A's response to the project deliverables question.

How Test Will Be Performed: Configure the existing discussion thread and its participants. Using Playwright, authenticate as Instructor A and post the specified reply. Verify that the reply appears in the thread with the correct author and timestamp. Capture outgoing email using a controlled test email service and verify that Supervisor A receives a notification containing the event summary and valid discussion link, without confidential project information.

5. **T-FR-39: Notify Teams of Supervisor Application Decisions**

   Type: Automatic

   Initial State: Team Amazing Engineers has submitted an application to Autonomous Waste Sorting System with status "Applied". The supervisor has not made a decision, and Team Amazing Engineers's members have valid registered email addresses.

   Input: The supervisor accepts Team Amazing Engineers's application.

   Output: The application changes to "Supervisor Accepts". Relevant authorized members of Team Amazing Engineers receive an email notification identifying the supervisor's decision and providing a link to the application. No unrelated student receives the notification.

   Test Case Derivation: Derived from `FR_ThreadsNotify` and `FR_DeclineTeam`. The SRS requires notification of decisions on project applications and agreements. The scenario verifies that relevant users are notified following a supervisor decision.

   How Test Will Be Performed: Using Playwright, authenticate as the supervisor and accept the application. Capture outgoing emails using the test email service. Verify the event summary, application link, recipient, and corresponding application status.

6. **T-FR-40: Notify Students When a Project Becomes Available**

   Type: Automatic

   Initial State: Instructor A is authorized to review proposals for the Software Engineering capstone course. The Autonomous Waste Sorting System proposal has status "Proposal submitted" and has not yet been published. Student A is enrolled in the course.

   Input: Instructor A approves the proposal, making the project available to students.

   Output: The project becomes "Open" and is published in the approved project catalog. Student A receives an email notification identifying that a new project listing is available and providing a link to the listing.

   Test Case Derivation: Derived from `FR_ThreadsNotify` and `FR_ReviewProposal`. The SRS specifies that students receive notifications about new project listings. Approving and publishing a proposal provides a concrete event for verifying this behaviour.

   How Test Will Be Performed: Configure the submitted proposal and enrolled student. Using Playwright, authenticate as Instructor A and approve the proposal. Verify the project appears in the catalog and that the test email service captures the notification sent to Student A.

7. **T-FR-41: Exclude Confidential Information from Email Notifications**

Type: Automatic

Initial State: Team Amazing Engineers has an application to Autonomous Waste Sorting System. The project contains the confidentiality statement "Restricted Prototype Specifications" and an IP field containing "Confidential Patent Agreement". The supervisor and Team Amazing Engineers are authorized to receive notifications about the application.

Input: The supervisor accepts Team Amazing Engineers's application, triggering an email notification.

Output: The email contains a summary of the acceptance decision and a link to the application. It does not contain either protected value:

- Restricted Prototype Specifications
- Confidential Patent Agreement

It also does not include sensitive details about the project.

Test Case Derivation: Derived from `FR_ThreadsNotify`, which requires notification emails to exclude confidential, IP, and other sensitive project details. The system's response to detecting senstive values determines whether protected information was disclosed.

How Test Will Be Performed: Populate the corresponding records with the specified values. Using Playwright, trigger the supervisor acceptance and capture the resulting email. Inspect the subject and body, including any generated preview content. Verify the event summary and link are present and that none of the protected values appear in the email.

8. **T-FR-42: Exclude Unauthorized Users from Notification Recipients**

Type: Automatic

Initial State: External Supervisor A owns Autonomous Waste Sorting System. External Supervisor B owns a different project and has no association with Supervisor A's project. Team Amazing Engineers has an application to Supervisor A's project. Supervisor B is not authorized to view that project or its application.

Input: Supervisor A accepts Team Amazing Engineers's application.

Output: Authorized recipients associated with the application receive the relevant email notification. Supervisor B receives no notification about Supervisor A's project, and no project information is exposed to Supervisor B.

Test Case Derivation: Derived from `FR_ThreadsNotify` and `FR_ExternalSupervisorScope`. Notification recipients must be determined according to their authorization at the time of sending. The test verifies that project confidentiality is maintained through email notifications.

How Test Will Be Performed: Configure both supervisors and Team Amazing Engineers's application. Using Playwright, trigger the acceptance decision. Inspect the outgoing email records and verify that only authorized recipients receive project-related notifications. Confirm that Supervisor B's was not sent any messages related to the event.

9. **T-FR-43: Re-evaluate Notification Recipients After an Authorization Change**

Type: Automatic

Initial State: Student A and Student B are members of Team Amazing Engineers, which has an application to Autonomous Waste Sorting System. Both students initially have access to the associated discussion thread. Student B is subsequently removed from the team through an authorized membership change before a new notification is generated.

Input: The project supervisor posts a reply to Team Amazing Engineers's application discussion thread after Student B's removal.

Output: Student A receives the new reply notification. Student B, who is no longer authorized to access the application discussion, does not receive a notification about the new reply.

Test Case Derivation: Derived from `FR_ThreadsNotify`, which requires recipients to be determined when the notification is sent. Removing a user's authorization before generating a notification tests that the system uses current permissions rather than a previously stored recipient list.

How Test Will Be Performed: Configure Team Amazing Engineers and the application discussion. Remove Student B using the supported team membership workflow. Using Playwright, authenticate as the supervisor and post the specified reply. Inspect outgoing emails and verify that it is sent to Student A but not Student B. Confirm that Student B cannot access the application thread.

4.1.7 Team-Project Approval and Validation

This section covers `FR_RequestApproval`, `FR_ApprovalResponse`, `FR_ApprovalPreflightValidate`, `FR_ExternalApproval`, and `FR_ApprovalRevision` in Section 6 of the SRS, along with the team-project status lifecycle defined in Section 7.2 and Figure 2. The tests verify that student teams can submit agreed projects for approval, that submissions are validated against course rules, and that instructors and TAs can approve, reject, or request modifications. They also verify external approval recording and the handling of changes to previously approved projects.

1. **T-FR-44: Submit an Agreed Team-Project for Approval**

   Type: Automatic

   Initial State: The Software Engineering capstone course has in-platform approval enabled. Team Amazing Engineers consists of four eligible students and has reached an agreement with the supervisor of Autonomous Waste Sorting System. Its team-project status is "Agreed". The team satisfies the course's minimum and maximum team sizes of 3 and 5 students. All project and team information required by the instructor's configuration for the Software Engineering capstone course has been provided, including any course-specific required fields.

   Input: An authorized member of Team Amazing Engineers submits the agreed project and team information for instructor approval.

   Output: The submission is accepted and its status changes from "Agreed" to "Team project submitted". The submitted project and team information is recorded and becomes available to the course instructor for review.

   Test Case Derivation: Derived from `FR_RequestApproval` and `FR_ApprovalPreflightValidate`. The team has a valid agreement, complete information, and a membership count satisfying

$3 \leq n \leq 5$, since $n = 4$. The submission therefore satisfies the specified preflight conditions and should enter the approval process.

How Test Will Be Performed: Configure the agreed team-project with complete information and enable in-platform approval. Using Playwright, authenticate as a member of Team Amazing Engineers and submit the team-project for approval. Verify the updated status, saved submission information, and visibility in the instructor's approval interface.

2. **T-FR-45: Reject Approval Submissions with Incomplete Information**

   Type: Automatic

   Initial State: The Software Engineering capstone course has in-platform approval enabled. Team Amazing Engineers consists of four valid members and has an agreed project titled Autonomous Waste Sorting System. All required team-project information is initially complete.

   Input: Repeat the approval submission test while omitting one required project or team information field at a time.

   Output: Each incomplete submission is rejected with a validation message identifying the missing information. The application remains in its previous status, and no submission is forwarded for instructor review.

   Test Case Derivation: Derived from `FR_ApprovalPreflightValidate`, which requires submissions to contain complete project information. For the set of required fields, a submission is complete only when all fields are valid. Omitting any required field violates this condition and must prevent submission.

   How Test Will Be Performed: Prepare an otherwise valid agreed team-project. Using Playwright, remove each required field individually and attempt submission. Verify that the corresponding validation message appears, no approval submission is created, and the original application status remains unchanged.

3. **T-FR-46: Reject Approval Submissions Violating Course Rules**

   Type: Automatic

   Initial State: The Software Engineering capstone course requires teams of 3 to 5 students and permits teams consisting of Software Engineering students. In-platform approval is enabled. Team Amazing Engineers has an agreed project and complete project information.

   Input: Attempt to submit Team Amazing Engineers's project for approval under the following independent membership configurations:

   - 2 Software Engineering students.
   - 3 Software Engineering students.
   - 5 Software Engineering students.
   - 6 Software Engineering students.

   Output: Submissions with 2 or 6 members are rejected. Submissions with 3 or 5 members pass the team-size preflight check, provided all other requirements are satisfied. Rejected submissions do not enter the instructor review process.

Test Case Derivation: Derived from `FR_ApprovalPreflightValidate` and `FR_ValidateTeamRules`. For team size n, the configured validity constraint is

$$3 \leq n \leq 5.$$

Therefore, $n = 2$ and $n = 6$ must fail validation, while $n = 3$ and $n = 5$ satisfy the boundary conditions. This verifies that approval preflight checks enforce the current course rules.

How Test Will Be Performed: Independently configure the four team membership scenarios using test fixtures. For each scenario, authenticate as an authorized team member and attempt submission. Verify the expected acceptance or rejection and ensure rejected submissions do not create instructor review records.

4. **T-FR-47: Approve a Submitted Team-Project In-Platform**

   Type: Automatic

   Initial State: In-platform approval is enabled. Team Amazing Engineers has submitted Autonomous Waste Sorting System for approval. The application has status "Team project under review", and all project and team information satisfies the applicable course rules. Instructor A is authorized to review the submission.

   Input: Instructor A selects "Approve" for Team Amazing Engineers's submitted project.

   Output: The team-project status changes to "Approved". The approval decision is recorded, and the project and team are identified as approved in the relevant course records. The project's reserved capacity remains unchanged.

   Test Case Derivation: Derived from `FR_ApprovalResponse` and Section 7.2 of the SRS. The instructor's approval completes the review process without creating an additional supervisor reservation. The number of reserved slots on the project remains the same. This verifies that team-project approval changes the approval status without incorrectly modifying project capacity.

   How Test Will Be Performed: Configure the submitted application under instructor review. Using Playwright, authenticate as Instructor A and approve the submission. Verify the "Approved" status, recorded decision, updated course information, and unchanged reserved-slot count.

5. **T-FR-48: Request Modifications to a Team-Project Submission In-Platform**

   Type: Automatic

   Initial State: Team Amazing Engineers has submitted Autonomous Waste Sorting System for instructor approval. The submission has status "Team project under review". Its project objectives state "Develop an automated waste sorting system". Instructor A is authorized to review the submission.

   Input: Instructor A selects "Request modifications" and provides the feedback: "Please clarify which types of waste the proposed system will identify and sort."

   Output: The team-project status changes to "Team project changes requested". The instructor's feedback is recorded and made available to Team Amazing Engineers. The submission is not marked approved.

Test Case Derivation: Derived from `FR_ApprovalResponse` and Section 7.2 of the SRS. The requirement permits instructors to request modifications with feedback rather than immediately approving or rejecting a team-project submission.

How Test Will Be Performed: Using Playwright, authenticate as Instructor A, open Team Amazing Engineers's submission, and request the specified modifications. Verify the updated status, preserved feedback, and absence of an approval decision.

6. **T-FR-49: Revise and Resubmit a Team-Project After Feedback**

   Type: Automatic

   Initial State: Team Amazing Engineers's Autonomous Waste Sorting System submission has status "Team project changes requested". Instructor A previously requested clarification on the types of waste the proposed system will identify and sort. The submission currently lists "Develop an automated waste sorting system" as its project objective.

   Input: An authorized Team Amazing Engineers member updates the project objective to "Develop an automated waste sorting system capable of identifying and sorting plastic and metal waste" and resubmits the team-project.

   Output: The revised information is saved, and the team-project returns to "Team project submitted" for instructor review. The previous feedback remains available, and the submission is not automatically approved.

   Test Case Derivation: Derived from `FR_RequestApproval` and `FR_ApprovalResponse`. The SRS allows teams to modify and resubmit their information following instructor feedback. The revised submission must re-enter the review process rather than bypassing approval.

   How Test Will Be Performed: Configure the change request and feedback. Using Playwright, authenticate as an authorized Team Amazing Engineers member, update the project objective, and resubmit. Verify the updated objective, submission status, and preservation of the previous feedback.

7. **T-FR-50: Reject a Submitted Team-Project**

   Type: Automatic

   Initial State: Team Amazing Engineers has submitted Autonomous Waste Sorting System for instructor approval. The team-project has status "Team project under review". The project has maximum capacity $C_{max} = 2$ and $R = 2$ reserved slots, one of which is held by Team Amazing Engineers. Instructor A is authorized to review the submission.

   Input: Instructor A rejects Team Amazing Engineers's submission with the feedback: "The proposed project scope does not meet the course's capstone requirements."

   Output: The team-project status changes to "Team project rejected". The rejection decision and feedback are recorded. Team Amazing Engineers's reserved slot is released, reducing R from 2 to 1. The project's status changes from "Full" to "Open".

   Test Case Derivation: Derived from `FR_ApprovalResponse` and the capacity rules in Section 7.2 of the SRS. Rejection of a team-project requires releasing its reserved slot. Initially,

$$C_{\text{remaining}} = 2 - 2 = 0.$$

After rejection,

$$R \leftarrow R - 1 = 1, \qquad C_{\text{remaining}} = 2 - 1 = 1.$$

The project therefore becomes "Open" again.

How Test Will Be Performed: Configure the project and submitted team-project with the specified capacity values. Using Playwright, authenticate as Instructor A and reject the submission. Verify the rejection status, recorded feedback, released reservation, and project availability.

8. **T-FR-51: Record an Approval Through an External Approval Process**

   Type: Automatic

   Initial State: The Software Engineering capstone course uses an external project approval process without an integration adapter. Team Amazing Engineers has an agreed project titled Autonomous Waste Sorting System, and its approval has been confirmed through the external process. Instructor A and TA A are authorized to record approval decisions for the course.

   Input: Instructor A manually records the external approval decision for Team Amazing Engineers's project.

   Output: The team's project approval status is updated to "Approved". The recorded decision is associated with the correct team and project and becomes visible in the course's approval records.

   Test Case Derivation: Derived from `FR_ExternalApproval`, which requires the system to support manual recording of externally determined approval decisions when no integration adapter is available. The test verifies that the platform can represent an approval obtained outside Keystone.

   How Test Will Be Performed: Configure the course to use external approvals without an adapter. Using Playwright, authenticate as Instructor A and record Team Amazing Engineers's approval. Verify the saved approval decision and its association with the correct team and project.

9. **T-FR-52: Preserve Original Approval During a Proposed Project Change**

   Type: Automatic

   Initial State: Team Amazing Engineers's Autonomous Waste Sorting System project has status "Approved". The approved project objective is "Develop a computer vision system capable of identifying and sorting plastic and metal waste". Instructor A is authorized to review changes to approved projects.

   Input: Team Amazing Engineers proposes changing the project objective to "Develop a computer vision system capable of identifying and sorting plastic, metal, and paper waste".

   Output: The proposed change is submitted for instructor review. The team-project status is displayed as "Approved (change under review)". The original approved objective remains effective until the instructor accepts the change. The proposed objective is retained separately for review without overwriting the approved record.

Test Case Derivation: Derived from `FR_ApprovalRevision` and Section 7.2 of the SRS. The requirement states that a previously approved project must retain its approval while a proposed modification awaits review. Therefore, the Objective does not change while the instructor acceptance is pending. Only an accepted revision may replace the original approved objective.

How Test Will Be Performed: Configure the approved team-project with its original objective. Using Playwright, authenticate as an authorized Team Amazing Engineers member and submit the proposed change. Verify the pending-change status, preservation of the original approved objective, and availability of the proposed objective for instructor review.

10. **T-FR-53: Successfully Synchronize Approval Through an External Adapter**

Type: Automatic

Initial State: The Software Engineering capstone course uses GitLab as its external approval system, connected to Keystone through an integration adapter. Team Amazing Engineers has submitted Autonomous Waste Sorting System for approval, with status "Team project submitted". The integration adapter is configured to associate this with the corresponding GitLab MR approval record.

Input: The integration adapter transfers Team Amazing Engineers's submission information to the corresponding GitLab approval record. An authorized reviewer approves the submission in GitLab. The adapter then synchronizes the approval decision back to Keystone.

Output: The GitLab approval record is associated with a Keystone submission id. Following synchronization, the team-project status changes to "Approved", and the external approval decision is recorded against Team Amazing Engineers's project along with a timestamp.

Test Case Derivation: Derived from `FR_ExternalApproval`, which requires Keystone to synchronize approval decisions through an available external integration adapter. A submission identifier establishes the correspondence between the two systems, ensuring the approval is applied to the correct team-project. Successful synchronization must update the approval status and timestamp.

How Test Will Be Performed: Configure a controlled GitLab integration environment and associate Keystone submission with its corresponding GitLab approval record. Using Playwright, submit the team-project through Keystone and verify that the adapter transfers the relevant information to GitLab. Record an approval decision in GitLab and trigger synchronization. Verify that the decision updates only the corresponding Keystone team-project to "Approved" and that the last successful synchronization timestamp is displayed.

11. **T-FR-54: Handle a Failed External Approval Synchronization**

Type: Automatic

Initial State: The Software Engineering capstone course uses an external approval system connected through an integration adapter. Team Amazing Engineers's Autonomous Waste Sorting System application has status "Team project under review". The previous successful synchronization occurred at 2026-10-30 14:00:00. The external approval service is configured to return a connection failure during the next synchronization.

Input: The integration adapter attempts to synchronize approval decisions with the unavailable external service.

Output: The synchronization failure is flagged by Keystone. Team Amazing Engineers's application remains "Team project under review", and no unverified approval decision is recorded. The last successful synchronization timestamp remains 2026-10-30 14:00:00.

Test Case Derivation: Derived from `FR_ExternalApproval`, which requires failed synchronization attempts to be flagged and the timestamp of the last successful synchronization to be displayed. Since no successful synchronization occurs, the previously verified approval state and timestamp must remain unchanged.

How Test Will Be Performed: Configure a controlled external test service to fail during synchronization. Trigger the adapter and verify that Keystone displays a failure indicator. Confirm that the team-project status remains unchanged, no approval decision is incorrectly recorded, and the last successful synchronization timestamp is preserved.

12. **T-FR-55: Accept a Revision to an Approved Team-Project**

    Type: Automatic

    Initial State: Team Amazing Engineers's Autonomous Waste Sorting System project has status "Approved (change under review)". The original approved objective is "Develop a computer vision system capable of identifying and sorting plastic and metal waste". The team has submitted a proposed objective: "Develop a computer vision system capable of identifying and sorting plastic, metal, and paper waste". Instructor A is authorized to review the proposed change.

    Input: Instructor A reviews and accepts the proposed revision.

    Output: The proposed objective replaces the original approved objective. The team-project status returns to "Approved", and the accepted change is recorded. The project remains approved throughout the revision process.

    Test Case Derivation: Derived from `FR_ApprovalRevision` and Section 7.2 of the SRS. Approved project information must remain effective until an instructor accepts the proposed change. Following acceptance, the approved objective becomes the revised objective. The status returns to "Approved" once the revision decision is completed.

    How Test Will Be Performed: Configure the approved project with the original objective and pending revision. Using Playwright, authenticate as Instructor A and accept the proposed change. Verify that the revised objective becomes the approved value, the status returns to "Approved", and the decision is recorded.

13. **T-FR-56: Reject a Revision to an Approved Team-Project**

    Type: Automatic

    Initial State: Team Amazing Engineers's Autonomous Waste Sorting System project has status "Approved (change under review)". The original approved objective is "Develop a computer vision system capable of identifying and sorting plastic and metal waste". The proposed revision would expand the objective to include paper waste. Instructor A is authorized to review the proposed change.

Input: Instructor A rejects the proposed revision with the feedback "The additional material category exceeds the approved project scope."

Output: The proposed revision is rejected, and the team-project status returns to "Approved". The original approved objective remains unchanged, and the rejection decision and feedback are recorded. The existing project approval remains valid.

Test Case Derivation: Derived from `FR_ApprovalRevision` and Section 7.2 of the SRS. Rejecting a proposed revision must preserve the previously approved information rather than replacing it. Therefore, the Objective does not change. The rejection affects only the proposed change, not the project's existing approval.

How Test Will Be Performed: Configure the approved project with its original objective and pending revision. Using Playwright, authenticate as Instructor A and reject the proposed change with the specified feedback. Verify that the original objective remains approved, the project status returns to "Approved", and the rejection decision and feedback are recorded.

4.1.8 Course Configuration, Reference Materials, and Administration

This section covers `FR_DisableFeatures`, `FR_RejectComponentDisable`, `FR_ReferenceMaterial`, `FR_RequiredActionsList`, `FR_ActionRequiredEmail`, `FR_ReadCourseInformation`, and `FR_ExportCourseInf` in Section 6 of the SRS. The tests verify that instructors can configure course features, authorized users can publish and access course resources, required actions are tracked and communicated, and course staff can view and export relevant administrative information. They also verify that configuration changes do not leave pending actions unresolved and that access is restricted to authorized users.

1. **T-FR-57: Configure Course Features**

Type: Automatic

Initial State: Instructor A manages the Software Engineering capstone course. Team formation, project proposals, and in-platform project approvals are enabled. No pending actions exist in these components, and the course configuration permits feature changes.

Input: Instructor A disables the in-platform project approval feature while leaving team formation and project proposals enabled.

Output: The configuration change is saved. In-platform project approval functionality is disabled for the Software Engineering course, while team formation and project proposal functionality remain enabled. The configuration of other courses remains unchanged.

Test Case Derivation: Derived from `FR_DisableFeatures`, which requires instructors to configure supported platform features independently for their courses. For each feature f, define its enabled state as $E(f) \in \{0, 1\}$. The expected configuration after the change is

$$E(\text{Approvals}) = 0, \qquad E(\text{Teams}) = 1, \qquad E(\text{Proposals}) = 1.$$

This verifies that disabling one feature does not unintentionally disable others.

How Test Will Be Performed: Using Playwright, authenticate as Instructor A and open the Software Engineering course configuration. Disable in-platform project approvals and

save the changes. Verify the saved configuration, confirm that the approval functionality is disabled, and ensure that team formation and project proposals remain available.

2. **T-FR-57A: Configure Required and Optional Project Proposal Fields**

   Type: Automatic

   Initial State: The Software Engineering capstone course has supervisor project proposals enabled. The course instructor is authenticated and has permission to configure proposal requirements. The proposal template contains fields for title, background, objectives, skills/requirements, project impact, and additional project terms.

   Input: The instructor configures title, background, objectives, and project impact as required fields, while skills/requirements and additional project terms are optional. A supervisor attempts to submit a proposal with the background field missing but all other required fields completed. The supervisor then supplies the background information and resubmits, leaving the optional fields empty.

   Output: The configuration is saved and applied to the course's proposal template. The first submission is rejected with a validation message identifying the missing background field. The second submission is accepted despite the optional fields being empty, and the proposal enters the "Proposal submitted" state.

   Test Case Derivation: Derived from `FR_ProposalOptionalFields`, `FR_SupervisorProposal`, and `FR_DisableFeatures`. The test verifies that the instructor's required and optional field configuration determines which proposal submissions satisfy the course's requirements.

   How Test Will Be Performed: Using Playwright, authenticate as the course instructor and configure the specified proposal fields as required or optional. Save the configuration and authenticate as a supervisor. Complete all required fields except background and attempt submission, verifying that the submission is rejected. Supply the missing background information while leaving optional fields empty and resubmit. Verify that the proposal is accepted with status "Proposal submitted".

3. **T-FR-57B: Configure Course-Specific Team-Project Approval Requirements**

   Type: Automatic

   Initial State: The Software Engineering capstone course has in-platform team-project approval enabled. Team Amazing Engineers consists of four eligible students and has reached an agreement with the supervisor of Autonomous Waste Sorting System. Its team-project status is "Agreed". The team satisfies the configured team formation rules. The course instructor is authenticated and authorized to configure approval submission requirements.

   Input: The instructor configures the approval process to require a project description, team information, and a public GitHub repository link, while making additional notes optional. Team Amazing Engineers attempts to submit its team-project information without the GitHub link. The team then supplies a valid GitHub link and resubmits, leaving additional notes empty.

   Output: The instructor's configuration is saved and applied to the Software Engineering capstone course. The first submission is rejected with a validation message identifying the

missing GitHub repository link, and the team-project status remains "Agreed". The second submission is accepted despite the optional notes being empty, and the team-project status changes to "Team project submitted".

Test Case Derivation: Derived from `FR_DisableFeatures`, `FR_RequestApproval`, and `FR_ApprovalPreflightValidate`. The test verifies that the platform enforces instructor-configured, course-specific submission requirements before permitting a team-project to enter the approval workflow.

How Test Will Be Performed: Using Playwright, authenticate as the instructor and configure the required approval submission fields. Save the configuration and authenticate as a member of Team Amazing Engineers. Complete the project description and team information but omit the required GitHub link. Attempt submission and verify that validation prevents the status transition. Enter a valid GitHub link, leave the optional notes empty, and resubmit. Verify that the submission succeeds and the team-project status becomes "Team project submitted".

4. **T-FR-58: Prevent Disabling a Component with Pending Actions**

   Type: Automatic

   Initial State: Instructor A manages the Software Engineering capstone course, which has in-platform project approvals enabled. Team Amazing Engineers has a submission with status "Team project under review", awaiting an instructor decision. No other approval submissions are pending.

   Input: Instructor A attempts to disable the in-platform project approval feature.

   Output: The configuration change is rejected. The system displays an error identifying the pending approval and shows Team Amazing Engineers's submission as an unresolved action. The approval feature remains enabled, and the submission retains its original status.

   Test Case Derivation: Derived from `FR_RejectComponentDisable`. A component cannot be disabled while it contains pending actions. The requested change must be rejected to prevent the pending submission from becoming inaccessible.

   How Test Will Be Performed: Configure the course with one pending team-project approval. Using Playwright, authenticate as Instructor A and attempt to disable the approval feature. Verify the error message and displayed pending submission. Confirm that the component remains enabled and the submission is unchanged.

5. **T-FR-59: Publish Course Reference Materials**

   Type: Automatic

   Initial State: The Software Engineering capstone course has no published FAQ titled "Capstone Project Deliverables". Instructor A is authorized to publish course resources. Student A is enrolled in the course, and External Supervisor A has an account authorized to read course reference materials.

   Input: Instructor A creates and publishes an FAQ titled "Capstone Project Deliverables" containing the question "What should a project proposal include?" and the answer "A project proposal should describe the objectives, expected deliverables, and required skills."

Output: The FAQ is saved and published under the Software Engineering course resources. Instructor A, Student A, and External Supervisor A can access the published resource and view its title, question, and answer.

Test Case Derivation: Derived from `FR_ReferenceMaterial`, which requires instructors and TAs to publish course resources for students and supervisors. The scenario verifies resource creation and that the resource is accessible by all relevant parties in the course.

How Test Will Be Performed: Using Playwright, authenticate as Instructor A and publish the specified FAQ. Verify that it appears in the course resources. Authenticate separately as Student A and External Supervisor A and confirm that both can retrieve the resource with its original content.

6. **T-FR-60: Update a Published Course Resource**

   Type: Automatic

   Initial State: The Software Engineering capstone course contains a published FAQ titled "Capstone Project Deliverables". Its answer states "Project proposals must include objectives and expected deliverables". TA A is assigned to the course and is authorized to maintain course resources.

   Input: TA A edits the FAQ answer to "Project proposals must include objectives, expected deliverables, and required skills" and saves the updated resource.

   Output: The updated FAQ is saved under the same course resource. Authorized students and supervisors see the revised answer when accessing it. The previous answer is no longer displayed as the current content.

   Test Case Derivation: Derived from `FR_ReferenceMaterial`, which requires course staff to maintain and edit existing resources. The test verifies that an authorized TA can update published material and that the updated version is accessible.

   How Test Will Be Performed: Configure the published FAQ. Using Playwright, authenticate as TA A, edit the answer, and save. Retrieve the resource as an enrolled student and as an authorized external supervisor. Verify the revised content and confirm that only one current FAQ record is displayed.

7. **T-FR-61: Display Required Actions and Completion States**

   Type: Automatic

   Initial State: Student A is enrolled in the Software Engineering capstone course and is a member of Team Amazing Engineers. The course requires the student to complete a profile and submit team-project information for instructor approval. Student A has completed their profile, but the team's approval submission has not yet been made.

   Input: Student A opens the required actions checklist.

   Output: The checklist displays "Complete student profile" as checked and "Submit team-project for approval" as unchecked. Both actions are associated with the correct student and course.

   Test Case Derivation: Derived from `FR_RequiredActionsList`, which requires users to view their assigned checklist items and completion states based on course requirements.

How Test Will Be Performed: Configure Student A's completed profile and pending team-project submission. Using Playwright, authenticate as Student A and open the checklist. Verify that both required actions are displayed with their expected completion states.

8. **T-FR-62: Automatically Update a Completed Checklist Action**

   Type: Automatic

   Initial State: Student A belongs to Team Amazing Engineers, which has reached an agreement with the supervisor of Autonomous Waste Sorting System. The Software Engineering capstone course has in-platform approval enabled. The required actions checklist shows "Submit team-project for approval" as incomplete. All submission information satisfies the course rules.

   Input: An authorized member of Team Amazing Engineers submits the agreed project and team information for instructor approval.

   Output: The team-project enters "Team project submitted" status. The checklist action "Submit team-project for approval" changes from incomplete to completed after the submission is verified. The checklist does not require a separate manual completion action.

   Test Case Derivation: Derived from `FR_RequiredActionsList` and `FR_RequestApproval`. The SRS requires checklist completion states to update when associated actions are completed and verified. This verifies consistency between the actual submission status and the corresponding checklist entry.

   How Test Will Be Performed: Configure the agreed team-project and incomplete checklist action. Using Playwright, authenticate as an authorized Team Amazing Engineers member and submit the project for approval. Verify the successful submission, refresh the checklist, and confirm that the action is marked completed.

9. **T-FR-63: Send Email Reminders for Outstanding Actions and Deadlines**

   Type: Automatic

   Initial State: The Software Engineering capstone course has administrative email reminders enabled. The course has a project approval submission deadline of November 15, 2026. Student A has an incomplete "Submit team-project for approval" checklist action. Student B has already completed the same action. The reminder is configured to run on November 12, 2026.

   Input: The configured administrative notification process runs on November 12, 2026.

   Output: Student A receives an email identifying the outstanding approval submission and the November 15 deadline. Student B is not incorrectly identified as having an incomplete approval submission.

   Test Case Derivation: Derived from `FR_ActionRequiredEmail`, which requires configured email notifications for upcoming deadlines and outstanding checklist actions. The reminder occurs three calendar days before the deadline. The scenario verifies that an incomplete required action is identified in a configured reminder without misclassifying a completed action.

   How Test Will Be Performed: Configure the reminder, deadline, and student checklist states. Run the notification process using a controlled test clock and capture outgoing email

through a test email service. Verify that Student A's reminder identifies the correct action and deadline, and that Student B is not described as having the same outstanding action.

10. **T-FR-64: View and Filter Course-Wide Administrative Information**

    Type: Automatic

    Initial State: Instructor A manages the Software Engineering capstone course. Its records include:

    - Project A: Autonomous Waste Sorting System, "Open", with Team Amazing Engineers' application under instructor review.
    - Project B: Smart Campus Navigation, "Open", with Team Fantastic Coder's project approved.
    - Project C: Automated Assembly Line, "Full", with Team Capstone Legends' project approved.

    Each project has an assigned supervisor, and the associated teams have valid membership records.

    Input: Instructor A opens the course administration overview and filters team-project records by approval status "Approved".

    Output: The filtered results contain Team Fantastic Coders and Team Capstone Lengends, but exclude Team Amazing Engineers, whose submission is still under review. The instructor can view the associated projects, team compositions, supervisors, and approval statuses.

    Test Case Derivation: Derived from `FR_ReadCourseInformation`, which requires instructors to access and filter project and team information across their assigned courses. Searching for approved projects records produce exactly two matching team-projects.

    How Test Will Be Performed: Populate the course with the specified records. Using Playwright, authenticate as Instructor A and apply the "Approved" approval-status filter. Verify that exactly two approved team-projects appear and that their administrative details are accessible.

11. **T-FR-65: Restrict Instructor and TA Access to Authorized Courses**

    Type: Automatic

    Initial State: The Software Engineering capstone course contains Team Amazing Engineers and Team Fantastic Coders. Team Amazing Engineers works on Smart Campus Navigation, while Team Fantastic Coders works on Automated Assembly Line. Both teams have project records, approval statuses, and associated discussion threads. Instructor A and TA A are assigned to the Software Engineering capstone course, with TA A specifically assigned to Team Amazing Engineers. A separate Mechatronics capstone course contains Team Capstone Legends. Instructor A is also authorized to access the Mechatronics course, while TA A is assigned only to the Software Engineering Course.

    Input: Instructor A and TA A each access the course administration interface and attempt to view the records of Team Amazing Engineers, Team Fantastic Coders, and Team Capstone Legends.

Output: Instructor A and TA A can access the project, team, approval status, and relevant discussion records of both Team Amazing Engineers and Team Fantastic Coders because both teams belong to their authorized course. TA A's assignment to Team Amazing Engineers does not prevent access to Team Fantastic Coders' records. Instructor A can access Team Capstone Legends' records since they are authorized to oversee the Mechatronics capstone course. However, TA A cannot access Team Capstone Legends' administrative records because they are not authorized to access the Mechatronics capstone course.

Test Case Derivation: Derived from `FR_ReadCourseInformation` and `FR_AuthorizationRBAC`. Instructors and TAs must be able to access administrative information across their authorized courses while being restricted from accessing records belonging to unauthorized courses.

How Test Will Be Performed: Configure the courses, teams, projects, and user assignments as specified. Using Playwright, authenticate as Instructor A and verify access to Team Amazing Engineers and Team Fantastic Coders, including their project statuses and relevant discussions. Verify that Team Capstone Legends' administrative records are accessible as well. Repeat the procedure as TA A and confirm that both Software Engineering teams are accessible despite TA A's individual assignment to Team Amazing Engineers. Attempt to access Team Capstone Legends' records directly as TA A and verify that access is denied.

12. **T-FR-66: Generate and Export a Course Administration Report**

Type: Automatic

Initial State: Instructor A manages the Software Engineering capstone course. The course contains the following records:

- Team Fantastic Coders: Students A, B, and C; approved project "Smart Campus Navigation"; Supervisor A.
- Team Capstone Legends: Students D, E, and F; approved project "Automated Assembly Line"; Supervisor B.
- Team Amazing Engineers: Students G, H, I, and J; project "Autonomous Waste Sorting System" currently under instructor review; Supervisor C.

Input: Instructor A requests an administrative report containing approved projects, team compositions, project selections, and supervisor information, then downloads the report.

Output: The system generates a downloadable report containing the two approved team-project records with their corresponding members, project titles, and supervisors. Team Amazing Engineers is correctly identified as under review. All exported information matches the corresponding course records.

Test Case Derivation: Derived from `FR_ExportCourseInformation`, which requires automatically generated, exportable administrative reports. The scenario verifies that report generation correctly includes requested information only which in this case is approved projects.

How Test Will Be Performed: Configure the three team-project records. Using Playwright, authenticate as Instructor A and generate the administrative report. Download and inspect the generated file using an automated file parser appropriate to its export format. Verify the presence of Team Fantastic Coders and Team Capstone Legends, their members and supervisors, and Team Amazing Engineers is not present.

4.1.9 Enrollment and Team Administration

This section covers `FR_ImportEnrollment`, `FR_UpdateEnrollment`, `FR_EnrollmentDiscrepency`, `FR_NeedsGroup`, and `FR_AssignTA` in Section 6 of the SRS. The tests verify that instructors and TAs can maintain course enrollment records, identify invalid or missing entries, locate students without teams, and assign TAs to teams. They also verify that enrollment changes do not occur without the required confirmation.

1. **T-FR-67: Import and Confirm a Course Enrollment List**

   Type: Automatic

   Initial State: Instructor A manages the Software Engineering capstone course. The current enrollment contains Students A and B. An Avenue to Learn enrollment list contains Students A, B, C, and D, each with a valid and unique student identifier. No import has been applied yet.

   Input: Instructor A imports the Avenue enrollment list, reviews the preview, and confirms the import.

   Output: Before confirmation, the preview identifies Students C and D as additions, and the existing enrollment remains unchanged. After confirmation, the course enrollment contains Students A, B, C, and D without duplicate records.

   Test Case Derivation: Derived from `FR_ImportEnrollment` and `FR_ReverifyUsers`. The SRS requires a preview and explicit confirmation before applying enrollment changes. Let E be the original enrollment and I the imported list:

   $$E = \{A, B\}, \qquad I = \{A, B, C, D\}.$$

   The expected additions are

   $$I \setminus E = \{C, D\}.$$

   Thus, the enrollment count remains 2 before confirmation and becomes 4 after confirmation.

   How Test Will Be Performed: Configure the existing enrollment and the Avenue enrollment source. Using Playwright, authenticate as Instructor A and initiate the import. Verify the preview identifies Students C and D and that no records have changed. Complete re-verification and confirm the import. Verify all four unique students are enrolled.

2. **T-FR-68: Reject Invalid Enrollment Import Entries**

   Type: Automatic

   Initial State: The Software Engineering capstone course contains Students A and B. Instructor A is authorized to import enrollment lists. A test enrollment file contains the following identifiers:

   - Student C: “400000001”.
   - Student D: “400000002”.

- Duplicate entry for Student C: "400000001".
- Student E: "INVALID-ID".

The test configuration recognizes the first two identifiers as valid and the final identifier as malformed.

Input: Instructor A uploads the enrollment file and attempts to confirm the import without correcting the identified problems.

Output: The import preview flags the duplicate identifier and invalid identifier. Confirmation is blocked until the problems are resolved. The original course enrollment remains unchanged.

Test Case Derivation: Derived from `FR_ImportEnrollment`, which requires duplicate and malformed identifiers to be identified before confirmation. A valid import requires all entries to satisfy the configured identifier rules and contain no duplicate identifiers.

How Test Will Be Performed: Prepare the enrollment file with the specified entries. Using Playwright, upload the file and inspect the preview. Verify that both problems are displayed and confirmation cannot proceed. Confirm that no enrollment records are added or removed.

3. **T-FR-69: Preserve Students Missing from an Enrollment Import**

Type: Automatic

Initial State: The Software Engineering capstone course contains Students A, B, C, and D. Students A, B, and C belong to Team Amazing Engineers, while Student D has no team. A new Avenue enrollment list contains Students A, C, and D but omits Student B.

Input: Instructor A imports the new enrollment list and reviews the enrollment discrepancies.

Output: Student B is flagged for instructor review because they are absent from the latest enrollment list. Student B remains enrolled and remains a member of Team Amazing Engineers until course staff explicitly confirm their removal.

Test Case Derivation: Derived from `FR_EnrollmentDiscrepency`. The system must identify students missing from the latest enrollment list without automatically removing them. Let E be the existing enrollment and I the imported list:

$$E = \{A, B, C, D\}, \qquad I = \{A, C, D\}.$$

The discrepancy set is

$$E \setminus I = \{B\}.$$

Student B must therefore be flagged, but their enrollment and team membership must be preserved until confirmed removal.

How Test Will Be Performed: Configure the existing enrollment and team memberships. Using Playwright, import the updated list and inspect the preview. Verify that Student B is flagged. Confirm that Student B remains in the course and Team Amazing Engineers without a separate removal confirmation.

4. **T-FR-70: Manually Add a Student to Course Enrollment**

   Type: Automatic

   Initial State: The Software Engineering capstone course has no Avenue integration configured. Students A, B, and C are enrolled. Student D has recently joined the course but has no enrollment record in Keystone. Instructor A is authorized to maintain course enrollment.

   Input: Instructor A manually adds Student D using the valid student identifier "400000004" and assigns them to the Software Engineering capstone course.

   Output: Student D is added to the course enrollment with the correct identifier and course association. The existing student records remain unchanged, and Student D becomes available in the course's student administration interface.

   Test Case Derivation: Derived from `FR_UpdateEnrollment`, which requires instructors and TAs to add students manually, including students joining after the initial enrollment import. The system must support enrollment updates without relying on Avenue integration.

   How Test Will Be Performed: Configure the course without Avenue integration. Using Playwright, authenticate as Instructor A and manually add Student D. Verify that the student appears in the course enrollment with the correct identifier and association.

5. **T-FR-71: Identify Enrolled Students Without Teams**

   Type: Automatic

   Initial State: The Software Engineering capstone course has six enrolled students. Students A, B, and C belong to Team Amazing Engineers, Student D belongs to Team Fantastic Coders, and Students E and F are not assigned to any team. Instructor A is authorized to view course administration information.

   Input: Instructor A opens the list of students who still need a team. Student E then joins Team Fantastic Coders, and Instructor A refreshes the list.

   Output: Initially, the list contains exactly Students E and F. Following Student E's team assignment, only Student F remains on the list. Students already assigned to teams are excluded.

   Test Case Derivation: Derived from `FR_NeedsGroup`. Let E represent enrolled students and M represent students assigned to teams. The required list is

   $$N = E \setminus M.$$

   Initially, $N = \{E, F\}$. After Student E joins Team Fantastic Coders, $N = \{F\}$. This verifies that the list accurately reflects current membership.

   How Test Will Be Performed: Configure the specified enrollment and team associations. Using Playwright, authenticate as Instructor A and verify the initial list. Assign Student E to Team Fantastic Coders through the supported team workflow, refresh the list, and verify that only Student F remains.

6. **T-FR-72: Assign a TA to a Student Team**

   Type: Automatic

   Initial State: Team Amazing Engineers consists of four students and is associated with Autonomous Waste Sorting System. No TA is assigned to the team. Instructor A is authorized to assign TAs, and TA A is associated with the Software Engineering capstone course.

   Input: Instructor A assigns TA A to Team Amazing Engineers.

   Output: The assignment is saved. Team Amazing Engineers's members and project supervisor can view TA A as the assigned TA.

   Test Case Derivation: Derived from `FR_AssignTA`, which requires course staff to assign TAs to teams and display the assignment to the team's members and supervisor. The test verifies the transition from an unassigned team to a correctly assigned team.

   How Test Will Be Performed: Using Playwright, authenticate as Instructor A and assign TA A to Team Amazing Engineers. Verify the saved assignment. Authenticate as a team member and as the project supervisor and confirm that both can view TA A's assignment.

4.1.10 Authentication, Authorization, and System Integrity

This section covers `FR_MacSSO`, `FR_AuthorizationRBAC`, `FR_UpdateRoles`, `FR_ExternalSupervisorVerificat` `FR_ExternalSupervisorScope`, `FR_ReverifyUsers`, `FR_LogExternalErrors`, `FR_DisplayTimestamps`, and `FR_DurableNotifications` in Section 6 of the SRS. The tests verify authentication through trusted providers, role-based access, external supervisor verification, protection of sensitive operations, error reporting, record timestamps, and notification recovery following system interruptions.

1. **T-FR-73: Authenticate Users Through Trusted Identity Providers**

   Type: Automatic

   Initial State: Student A has an active McMaster account associated with the Software Engineering capstone course. External Supervisor A has an account with the configured trusted external authentication provider. Neither user has an active Keystone session.

   Input: Student A signs in through McMaster SSO. External Supervisor A separately signs in using the trusted external authentication provider.

   Output: Both users are authenticated through their respective providers and can access the Keystone functions permitted by their assigned roles and verification status. Neither user receives access outside their authorized scope.

   Test Case Derivation: Derived from `FR_MacSSO` and `FR_AuthorizationRBAC`. The system must authenticate university and external users through trusted identity providers before granting access to platform services.

   How Test Will Be Performed: Configure controlled test accounts with the corresponding identity providers. Using Playwright, execute each sign-in workflow. Verify successful authentication, the resulting account identity, and access to the functions permitted by each user's role.

2. **T-FR-74: Deny Course Access When User Role Is Unknown**

   Type: Automatic

   Initial State: User A has successfully authenticated through a trusted identity provider, but no valid course role can be determined for the account. The Software Engineering course contains student profiles, team records, and project information.

   Input: User A attempts to access the course's project catalog, student profiles, and team administration information.

   Output: Access to all course data is denied. No student, team, or project information is disclosed to User A.

   Test Case Derivation: Derived from `FR_AuthorizationRBAC`, which explicitly requires users with undetermined roles to receive no access to course data. If $r(u)$ represents the course role of user u, then

   $$r(u) = \text{undefined} \Rightarrow \text{CourseAccess}(u) = 0.$$

   This verifies that successful authentication alone does not authorize access.

   How Test Will Be Performed: Configure an authenticated user without a valid course role. Using Playwright, attempt to access the specified course interfaces and direct record links. Verify that all requests are denied without disclosing course data.

3. **T-FR-75: Update a User's Course Role and Permissions**

   Type: Automatic

   Initial State: Administrator A (is_admin set) is authorized to manage user roles for the Software Engineering capstone course. User B is associated with the course and currently has the "Student" role. User B cannot access instructor-only course configuration features.

   Input: Administrator A changes User B's assigned course role from "Student" to "TA", completing any required re-verification.

   Output: User B's course role is updated to "TA". User B gains access to functions permitted to TAs in their assigned course scope, but does not gain instructor-only or administrator-only privileges. The updated role is visible in the administration interface.

   Test Case Derivation: Derived from `FR_UpdateRoles`, `FR_AuthorizationRBAC`, and `FR_ReverifyUsers`. The system must allow authorized administrators to update roles and enforce permissions associated with the new role.

   How Test Will Be Performed: Using Playwright, authenticate as Administrator A and update User B's role after completing re-verification. Verify that the updated role is saved. Authenticate as User B and confirm access to an authorized TA function while verifying that instructor-only configuration remains inaccessible.

4. **T-FR-76: Require Re-verification Before a Sensitive Action**

   Type: Automatic

Initial State: Supervisor A has an authenticated Keystone session and owns a proposal titled Autonomous Waste Sorting System with status "Draft". The proposal contains valid information and has not been deleted.

Input: Supervisor A attempts to delete the proposal but does not successfully complete the requested identity re-verification.

Output: The system requests identity re-verification before performing the deletion. Because re-verification is not completed, the proposal is not deleted and its existing information remains unchanged.

Test Case Derivation: Derived from `FR_ReverifyUsers`, which requires additional identity verification before deleting proposals and other sensitive actions. The test verifies that an existing authenticated session alone is insufficient to authorize proposal deletion.

How Test Will Be Performed: Configure Supervisor A and the draft proposal. Using Playwright, authenticate as Supervisor A and initiate deletion. Verify the re-verification challenge appears. Cancel or fail the challenge and verify that the proposal remains accessible and unchanged.

5. **T-FR-77: Record and Report an External Provider Error**

   Type: Automatic

   Initial State: Keystone is connected to a trusted external authentication provider. Administrator A is authorized to view external-provider errors.

   Input: A controlled external authentication service returns a service-unavailable failure during an authentication request.

   Output: Keystone records the external-provider error and notifies the configured university IT personnel or course instructor. Administrator A can view the recorded failure in the external-provider error log.

   Test Case Derivation: Derived from `FR_LogExternalErrors`, which requires external-provider failures to be logged and reported to IT and course instructors. The failure must be identifiable as an external-provider error rather than being silently ignored.

   How Test Will Be Performed: Configure the controlled provider to return a service-unavailable error. Trigger an authentication request and verify the resulting error record. Authenticate as Administrator A and inspect the external-provider error log. Verify that the event is recorded and that a notification is generated for an authorized recipient.

6. **T-FR-78: Display Record Last-Updated Timestamps**

   Type: Automatic

   Initial State: Student A has a completed student profile and is authorized to update it. The profile currently lists "Python and Java" as its skills. Its last updated timestamp is set to 2026-10-30 14:00:00.

   Input: Student A updates the skills field to "Python, Java, and React" and saves the profile at time 2026-10-30 15:00:00.

   Output: The updated profile displays the revised skills and a last-updated timestamp of 2026-10-30 15:00:00. The displayed timestamp corresponds to the record's latest modification.

Test Case Derivation: Derived from `FR_DisplayTimestamps`, which requires displayed records to show their last-updated timestamps. For the configured test times,

$$T_{\text{after}} > T_{\text{before}}.$$

The expected timestamp difference is one hour, verifying that the profile displays the latest update rather than its previous modification time.

How Test Will Be Performed: Configure the profile and controlled test clock. Using Playwright, authenticate as Student A, modify the skills field, and save. Verify the revised profile content and displayed last-updated timestamp.

7. **T-FR-79: Recover a Pending Notification After System Interruption**

Type: Automatic

Initial State: Team Amazing Engineers has submitted Autonomous Waste Sorting System for approval. Instructor A is authorized to receive notifications concerning the submission. The notification service is configured with durable pending-notification storage and a controlled test email service.

Input: The system generates a notification for Instructor A about the new submission. Before email delivery is verified, the notification service is abruptly stopped and then restarted.

Output: The pending notification remains stored during the interruption and is delivered after the service restarts. Instructor A receives the notification no more than once, and the system records verified delivery.

Test Case Derivation: Derived from `FR_DurableNotifications`, which requires pending notifications to persist until delivery is verified and prevents duplicate delivery to the same recipient. Let $D(u, n)$ denote the number of verified deliveries of notification n to user u. The required condition is

$$D(u, n) \leq 1.$$

Following recovery, the expected delivery count for Instructor A is

$$D(\text{InstructorA}, n) = 1.$$

This verifies notification recovery and the prevention of duplicate delivery.

How Test Will Be Performed: Configure the submission and notification service. Trigger the notification, then terminate the service before delivery is verified. Restart the service and inspect the controlled email service and pending notification records. Verify successful delivery and confirm that exactly one email was received for the notification.

...

4.2 Tests for Nonfunctional Requirements

4.2.1 Usability

These tests fulfil usability-related non-functional requirements: `NFR_UsabilityFirstSuccess`, `NFR_LearnUnas`

Primary Use-Case Usability Tests

1. **T-NFR-USBLTY-1: First-Time Use of Primary Workflows**

   Type: Dynamic, Manual

   Initial State: The system is deployed with seeded projects, teams, course timelines, and proposals, and is open at the sign-in page on a laptop. Each participant has a test account for their role, and the student accounts belong to a seeded team. One seeded student is on a team but absent from the latest enrollment import, and that team stays above the minimum team size without them. The eight participants are four students, one TA, two internal supervisors (professors), and one course instructor. The number of students is high because new students will need to learn to use the platform each year while instructors will eventually become familiar with the platform. No participant is a member of the Keystone team or has used or seen a demonstration of Keystone.

   Input/Condition: Each participant receives their account credentials and a written prompt for one task. The supervisor also receives a sheet with the proposal content to enter (Table 5):

   - **Student**: Filter for Mechatronics projects, select a project, and express the team's interest in it. End state: the team's application shows the status "Applied".
   - **Supervisor**: Submit a project proposal with only the required fields filled. End state: the system shows a visual confirmation that the proposal is "Proposal under review".
   - **Course Instructor**: Review a submitted project proposal, provide feedback, and request changes. End state: the system shows a visual confirmation that the review was submitted, with a preview of the feedback.
   - **TA**: Open the team composition view, identify the student absent from the enrollment list, confirm their removal, and export the validated team composition. End state: the export file starts downloading.

   Output/Result: At least six of eight participants (75%) complete their relevant tasks within 5 minutes without assistance. Disqualifying assistance is any help with using Keystone, such as navigation, what a field means or if it is required. Disqualifying assistance does not include clarifying the prompt, fixing hardware or network issues.

   How test will be performed: Each participant completes the test individually, and the seeded data is reset before each session. Two team members run each session: a facilitator and a note-taker. The facilitator obtains consent and gives the participant the prompt and any additional information such as credentials or form values. The facilitator explains that the task is timed, but that the test judges how intuitive Keystone is, not the participant's skills. Timing starts when the participant finishes reading the prompt and stops when the end state is reached. Any time spent fixing hardware/network/credential issues or clarifying the prompt should be subtracted. The facilitator does not guide the participant and gives assistance only when asked. If disqualifying assistance is given, the attempt is counted as unsuccessful, but the participant continues to the end state so that all issues are recorded. The note-taker records, for each participant, the elapsed time, whether the end state was reached, any assistance given, errors and wrong turns, and comments. The participant then completes the survey in Appendix 6.2.

   Relevant NFRs: `NFR_UsabilityFirstSuccess`, `NFR_LearnUnassisted`

4.2.2 Installability

These tests fulfil installability-related non-functional requirements: `NFR_DeployUnassisted`.

1. **T-NFR-INST-1: Unassisted Deployment**

   Type: Dynamic, Manual

   Initial State: A Linux server with Docker installed, reachable over the network, with a self-signed TLS certificate and key on the server. The participant has a laptop with SSH access to the server, the deployment guide (Appendix 6.4), the server's SSH credentials, the release tag to deploy, and a sheet with the admin credentials to configure. The participant has IT or software experience, is not a Keystone team member, and has never deployed Keystone.

   Input/Condition: The participant follows the deployment guide to connect to the server over SSH, download Keystone, configure the admin credentials and TLS certificate, and start the system.

   Output/Result: Within 30 minutes and without assistance, the test admin loads Keystone over HTTPS, bypassing the self-signed certificate warning, and signs in with the configured admin credentials. Disqualifying assistance is any help with Keystone or the deployment guide. Fixing server, network, or credential problems is not disqualifying assistance.

   How test will be performed: A test admin waits on a separate laptop. A facilitator gives the participant the guide and credentials, and explains that the deployment is timed, but that the test judges how clear the guide is, not the participant's skills. Timing starts when the participant opens the guide and stops when the test admin signs in. Timing pauses while server, network, or credential problems are fixed. The facilitator gives assistance only when asked. If disqualifying assistance is given, the deployment is counted as unsuccessful, but the participant continues until the test admin signs in so that all issues are recorded. The facilitator records the elapsed time, assistance given, the steps where the participant got stuck, and unclear parts of the guide.

   Relevant NFRs: `NFR_DeployUnassisted`

4.2.3 Portability

These tests fulfil portability-related non-functional requirements: `NFR_Browsers`, `NFR_ScreenSize`, `NFR_Container`.

1. **T-NFR-PORT-1: Browser and Screen Size Test**

   Type: Dynamic, Automated

   Initial State: The system is deployed with a test account for each role and seeded data in which each role can view at least one record of every type shown on its pages.

   Input/Condition: Every page is opened in Chrome, Firefox, and Safari at desktop (1920 px × 1080 px), tablet (1024 px × 768 px), and mobile (430 px × 932 px) resolutions, signed in as each role that can access it.

   Output/Result: In every browser and at every resolution, no page has horizontal overflow, clipped controls, or controls smaller than 40 px × 40 px, and all UI tests pass.

How test will be performed: Any code change on the main branch triggers a GitHub Actions job that runs the Playwright UI tests and layout checks in Playwright's Chromium, Firefox, and WebKit browsers at each resolution. The job records the browser version, resolution, test results, and screenshots or logs for failures.

Relevant NFRs: `NFR_ScreenSize`, `NFR_Browsers`

2. **T-NFR-PORT-2: Docker Build and Start Test**

   Type: Dynamic, Automated

   Initial State: A machine with Docker installed and the repository checked out at the latest commit on the main branch.

   Input/Condition: The frontend, backend, and database images are built and started together with Docker Compose.

   Output/Result: All images build without errors, and the backend health check returns a success response within 1 minute of start.

   How test will be performed: Any code change on the main branch triggers a GitHub Actions job that runs the build, starts the containers, and polls the health check until it succeeds or 1 minute has passed. The job records the Docker version, build logs, and container logs, and is marked as failed if the test fails.

   Relevant NFRs: `NFR_Container`

4.2.4 Accessibility

These tests fulfil accessibility-related non-functional requirements: `NFR_AccessibilityWCAG`, `NFR_Compliance`

1. **T-NFR-ACC-1: Automated Accessibility Audit**

   Type: Dynamic, Automated

   Initial State: Same as **T-NFR-PORT-1**.

   Input/Condition: Every page, signed in as each role that can access it, including open dialogs and forms showing validation errors, is audited against the WCAG 2.0 Level A and AA rules in Chrome at the resolutions in **T-NFR-PORT-1**.

   Output/Result: No critical violations are reported on any page at any resolution.

   How test will be performed: Any code change on the main branch triggers a GitHub Actions job in which the Playwright UI tests run an axe-core audit, limited to the WCAG 2.0 Level A and AA rules, in Playwright's Chromium-based browser after each page or state is loaded. The job records the page, role, resolution, rule, and impact level of every violation, and is marked as failed if any violation is critical.

   Relevant NFRs: `NFR_AccessibilityWCAG`, `NFR_ComplianceAODA`

4.2.5 Performance

These tests fulfil performance-related non-functional requirements: `NFR_PerfTail`, `NFR_PeakUsers`.

1. **T-NFR-PERF-1: Peak Load Test**

   Type: Dynamic, Automated

   Initial State: The system is deployed on a server matching the deployment target. Seeded data contains 100 open projects with discussion threads, 85 teams, and 400 test accounts split by role as in Table 8.

   Input/Condition: The load is ramped up to 400 simulated users over 5 minutes and then held for 10 minutes. Each simulated user repeatedly performs a randomly chosen task, where Table 8 gives the probability of each task for the user's role, with a 10 seconds pause between tasks.

   Output/Result: The 95th percentile latency of all requests during the 10 minutes at full load is at most 5.0 seconds.

   How test will be performed: Before each milestone, a manually started GitHub Actions workflow runs Locust on a separate machine to generate the load. Locust records the latency of every request. The report records the 95th percentile latency overall and per task, the request rate, the error rate, and the server's CPU and memory use.

   Relevant NFRs: `NFR_PerfTail`, `NFR_PeakUsers`

4.2.6 Reliability

These tests fulfil reliability-related non-functional requirements: `NFR_Uptime`, `NFR_Recoverable`. They also fulfil the privacy and data retention requirement `DPR_DailyDataBackups`.

1. **T-NFR-REL-1: Uptime Monitoring**

   Type: Dynamic, Automated

   Initial State: The system is deployed for a demo.

   Input/Condition: The backend health check is requested every 1 minute from a different network, from deployment until the end of the demo.

   Output/Result: Excluding scheduled maintenance windows, at least 99% of checks return HTTP 200 within 10 seconds.

   How test will be performed: A monitoring tool (such as Uptime Kuma) on a separate machine on a different network sends the checks and records the time and result of every check. Scheduled maintenance windows are logged before they start. The report records the monitored period, the success percentage, and each failed check's time and response.

   Relevant NFRs: `NFR_Uptime`

2. **T-NFR-REL-2: Restore From Backup**

   Type: Dynamic, Manual

Initial State: The system is deployed with seeded data and an admin account.

Input/Condition: A backup is taken. All projects and teams are then deleted directly in the database, and the latest backup is restored from the interface as the admin.

Output/Result: The seeded projects and teams are present after the restore.

How test will be performed: A team member takes the backup, deletes the projects and teams, and restores as the admin. They record the backup restored, its timestamp, whether the seeded data is present, and any messages shown.

Relevant NFRs: `NFR_Recoverable`

3. **T-NFR-REL-3: Scheduled Backup Creation**

Type: Dynamic, Manual

Initial State: The system is deployed and has never been started. No backups exist.

Input/Condition: The system is started for the first time. After the initial backup is created, the server's system time is set to 1 minute before the next backup is due, 24 hours after the initial backup.

Output/Result: An initial backup is created when the system first starts, and a second backup is created within 2 minutes of the time change.

How test will be performed: A team member starts the system and checks that the initial backup exists. They change the server's system time, wait 2 minutes, and check that a second backup exists. They record the timestamps of both backups.

Relevant NFRs: `NFR_Recoverable`

Relevant DPRs: `DPR_DailyDataBackups`

4.2.7 Maintainability

These tests fulfil maintainability-related non-functional requirements: `NFR_LiveDisable`, `NFR_ChangeEffort`.

1. **T-NFR-MAINT-1: Live Feature Disable**

Type: Dynamic, Automated

Initial State: The system is deployed and running with a seeded course in which every major feature is enabled: team formation, project proposals, project approvals, discussion threads, and recommendations. Test accounts exist for the course instructor and for each role that uses these features.

Input/Condition: For each major feature:

(a) An authenticated request that uses the feature is sent.

(b) The feature is disabled for the course through an authenticated request as the course instructor.

(c) The same request as in step 1 is sent.

Output/Result: For every feature, the request in step 1 returns its expected response, and the request in step 3 returns HTTP 403.

How test will be performed: Any code change on the main branch triggers a GitHub Actions job that runs a script sending these requests to the backend. The script records, for each feature, the requests sent and the status codes and responses received. The job is marked as failed if any feature does not give the expected result.

Relevant NFRs: `NFR_LiveDisable`

2. **T-NFR-MAINT-2: Likely Change Effort**

Type: Dynamic, Manual

Initial State: The components affected by the changes in Table 9 are implemented on the main branch. The table lists more changes than are expected in one week of maintenance.

Input/Condition: One of the 5 changes in Table 9 (20% of the week) is selected and implemented, including its tests.

Output/Result: The time to implement the change, multiplied by 5, is at most 8 hours.

How test will be performed: Before the final demo, a team member selects one of the changes, creates a branch from main, and implements it. Timing starts when work begins and stops when all CI checks pass. The team member opens a pull request for the branch, which is closed without merging and kept as the record. They record the change selected, its time, and the extrapolated weekly time. The overestimated week compensates for the team member's familiarity with the code.

Relevant NFRs: `NFR_ChangeEffort`

4.2.8 Security

These tests fulfil security-related non-functional requirements: `NFR_Authenticate`, `NFR_Authorize`, `NFR_ConcurrencySafety`, `NFR_EncryptTransit`, `NFR_EssentialPersonal`, `NFR_EncryptRest`, `NFR_Password` `NFR_AuditLog`, `NFR_AuditTamper`. They also fulfil the privacy and data retention requirements `DPR_RoleBasedDataPermissions`, `DPR_VisibilityMatrix`, `DPR_TLSCommunication`, `DPR_EncryptionAtRest` `DPR_NoPlaintextPasswords`.

1. **T-NFR-SEC-1: Unauthenticated Requests**

Type: Dynamic, Automated

Initial State: The system is deployed and running.

Input/Condition: An HTTP request is sent to each non-login endpoint with no credentials, authentication data, session, or token.

Output/Result: Every request returns an empty HTTP 401 response.

How test will be performed: Any code change on the main branch triggers a GitHub Actions job that runs a script against the backend. The script reads every endpoint and method from the backend's schema and sends the requests. It records each endpoint, method, status code, and response body. The job is marked as failed if any request does not give the expected result.

Relevant NFRs: `NFR_Authenticate`

2. **T-NFR-SEC-2: Role Access Matrix**

   Type: Dynamic, Automated

   Initial State: The system is deployed and running with two seeded courses. The first course has one account for each role (student, internal/external/unverified supervisor, TA, and course instructor), and at least one record of every type and state in the SRS tables "Visibility by record type and state" and "What an external supervisor can read" (Patel et al., 2026f). The second course has its own accounts and records. One account is signed in but not enrolled in any course.

   Input/Condition: As each account, a read request and a modify request are sent for every seeded record, including list, search, and count requests that return the record, and for one record ID of each type that does not exist.

   Output/Result: Read requests follow the two SRS visibility tables, and modify requests follow `DPR_RoleBasedDataPermissions`. Every allowed request succeeds. Every other request returns HTTP 404, the same response as a request for a record that does not exist, and the record does not appear in any list, search, or count. Accounts from the second course and the unenrolled account cannot read or modify any record in the first course.

   How test will be performed: Any code change on the main branch triggers a GitHub Actions job that runs a script sending these requests to the backend. The expected result for each account, record, and request is written as a table in the test code from the SRS tables. The script records each request, the expected result, and the received status code and response. The job is marked as failed if any request does not give the expected result.

   Relevant NFRs: `NFR_Authorize`

   Relevant DPRs: `DPR_RoleBasedDataPermissions`, `DPR_VisibilityMatrix`

3. **T-NFR-SEC-3: Concurrent Modifications**

   Type: Dynamic, Automated

   Initial State: The system is running with a seeded course containing:

   - A project with one free slot and 5 teams that have applied to it,
   - A team with one free spot and 5 students who have requested to join it,
   - A team with an application awaiting final approval.

   Input/Condition: Each of the following batches of requests is sent, with the requests in a batch sent at the same time:

   (a) The supervisor accepts all 5 teams for the project.

   (b) The team accepts all 5 students.

   (c) The team withdraws its application while the instructor approves it.

Output/Result: After every batch, the final state is equal to the state produced by running the same requests one at a time in some order. In particular, the project has exactly one accepted team, the team has exactly one new member, and the application is either withdrawn or approved, not both. Every request that was not applied returns an error response.

How test will be performed: Any code change on the main branch triggers a GitHub Actions job that runs the backend against a PostgreSQL database in Docker and sends the batches. The database is reset to the initial state before each batch, and each batch is repeated 50 times. After each batch, the job reads the affected records from the database and compares them with the possible serial outcomes. It records each batch, the responses received, and the final state. The job is marked as failed if any final state is not a possible serial outcome.

Relevant NFRs: `NFR_ConcurrencySafety`

4. **T-NFR-SEC-4: Unencrypted Requests**

   Type: Dynamic, Automated

   Initial State: The system is deployed and running, configured as in the deployment guide (Appendix 6.4).

   Input/Condition: An HTTP request for the home page, the sign-in endpoint, and a data endpoint is sent to port 80.

   Output/Result: Server responds only with a redirect to the same path over HTTPS.

   How test will be performed: Any code change on the main branch triggers a GitHub Actions job that starts the system with Docker Compose and sends the requests to port 80 from a separate container. The job records the status code, headers, and body received for each request. The job is marked as failed if any request does not give the expected result.

   Relevant NFRs: `NFR_EncryptTransit`

   Relevant DPRs: `DPR_TLSCommunication`

5. **T-NFR-SEC-5: Encrypted Data at Rest**

   Type: Dynamic, Manual

   Initial State: The system is deployed and running with seeded data. A student profile and a project are created with unique marker strings in their text fields, and a backup is created afterwards.

   Input/Condition: The database storage files and the latest backup are copied to a separate machine that does not have the decryption key. The copies are searched for the marker strings and opened with a new PostgreSQL instance and the standard restore tools. The backup is then restored to a new Keystone deployment on that machine.

   Output/Result: No marker string is found in any copied file, and the copies cannot be opened or restored to readable data. Restoring the backup to the new deployment prompts for the decryption key and does not restore any data without it. With the correct key, the restore succeeds and the marker strings are present.

   How test will be performed: A team member with access to the server copies the files, searches them (for example with `grep`), and tries to open and restore them. They then deploy

Keystone on the separate machine following the deployment guide (Appendix 6.4) and restore the backup from the interface as the admin, first without the key and then with it. They record the files copied, the search commands and their results, and the result of each attempt to open or restore the copies.

Relevant NFRs: `NFR_EncryptRest`

Relevant DPRs: `DPR_EncryptionAtRest`

6. **T-NFR-SEC-6: Password Storage**

   Type: Dynamic, Automated

   Initial State: The system is running with no user accounts.

   Input/Condition: Two local accounts (an admin and an external supervisor) are created with the same password.

   Output/Result: Both stored values are Argon2id hashes with a random salt (starting with `$argon2id$`), neither contains the password, the two values have different salts, and the two hashes are different.

   How test will be performed: Any code change on the main branch triggers a GitHub Actions job that runs the backend against a PostgreSQL database in Docker, creates the accounts, and reads both stored password values directly from the database. The job records both stored values and the result of each check. The job is marked as failed if any check fails.

   Relevant NFRs: `NFR_PasswordHash`

   Relevant DPRs: `DPR_NoPlaintextPasswords`

7. **T-NFR-SEC-7: Audit Log Replay**

   Type: Dynamic, Automated

   Initial State: The system is running with an empty database, except for accounts for every role.

   Input/Condition: Every type of approval decision, status change, and role change is performed at least once, as each role that can perform it. The audit log entries are then applied in order to a second database with the same accounts and no other data. Once every user action is logged, the scenario is extended to every type of user action.

   Output/Result: The approval decisions, statuses, and roles in the second database are equal to those in the first. Every log entry records its time, the action, the affected record, the user, and the user's role at the time of the action.

   How test will be performed: Any code change on the main branch triggers a GitHub Actions job that runs the backend against a PostgreSQL database in Docker, performs the actions with a scripted scenario, and applies the log entries with a replay script. The job compares the approval decisions, statuses, and roles in both databases and records every difference found. The job is marked as failed if any difference is found or any log entry is missing a required field.

   Relevant NFRs: `NFR_AuditLog`

8. **T-NFR-SEC-8: Audit Log Tampering**

   Type: Dynamic, Manual

   Initial State: The system is deployed and running with seeded data and at least 100 audit log entries. Each entry includes an HMAC of the entry and the previous entry's HMAC, computed with a key held by the key management provider or hardware security module, not stored in the database. An operator email address is configured.

   Input/Condition: Each of the following is done directly on the audit log storage, with the storage restored before the next:

   (a) The content of one entry is changed.

   (b) One entry is deleted.

   (c) The content of one entry is changed, and the HMAC of that entry and every later entry is recalculated without the key (for example with plain SHA-256).

   (d) The audit log storage is deleted.

   Output/Result: Each change raises an alert to the operator email address within 5 minutes that identifies the affected storage.

   How test will be performed: This test is run once tamper-evident storage is implemented. A team member connects to the database as the database administrator (for example with `psql`), makes each change with SQL, and restores the storage afterwards. They record the time of each change, the time each alert is received, and its content.

   Relevant NFRs: `NFR_AuditTamper`

9. **T-NFR-SEC-9: Personal Data Inspection**

   Type: Static, Inspection

   Who is involved: Two team members

   When does it occur: Before the Revision 0 and final demos.

   How it will be performed: Each inspector independently lists every `table.column` in the deployed database that stores personal information, and maps each one to an FR or NFR in the SRS (Patel et al., 2026f). Differences between the two lists, and between the two mappings, are resolved by consensus. Any column that is not mapped has a GitHub issue tracking it.

   Pass Condition: Every listed column is mapped to an FR or NFR.

   Relevant NFRs: `NFR_EssentialPersonal`

4.3 Traceability Between Test Cases and Requirements

Functional Requirement	Test Cases
`FR_SupervisorProposal`	T-FR-01, T-FR-57A

FR_ProposalOptionalFields	T-FR-02, T-FR-03, T-FR-57A
FR_StudentProposal	T-FR-04
FR_ReviewProposal	T-FR-05, T-FR-06, T-FR-07, T-FR-08, T-FR-40
FR_ViewApprovedProjects	T-FR-09, T-FR-10
FR_ViewProjectStatus	T-FR-11, T-FR-12
FR_StudentProfile	T-FR-13, T-FR-14
FR_StudentTeams	T-FR-15, T-FR-16, T-FR-17, T-FR-18
FR_ValidateTeamRules	T-FR-19, T-FR-20, T-FR-21, T-FR-22, T-FR-46
FR_TeamDiscovery	T-FR-23
FR_TeamProfile	T-FR-24
FR_TeamRecommend	T-FR-25
FR_ProjectInterest	T-FR-26
FR_DeclineTeam	T-FR-27, T-FR-28, T-FR-29, T-FR-30, T-FR-31, T-FR-32, T-FR-33, T-FR-34, T-FR-39
FR_ProjectThreads	T-FR-35, T-FR-36, T-FR-37
FR_ThreadsNotify	T-FR-38, T-FR-39, T-FR-40, T-FR-41, T-FR-42, T-FR-43
FR_RequestApproval	T-FR-44, T-FR-49, T-FR-57B, T-FR-62
FR_ApprovalResponse	T-FR-47, T-FR-48, T-FR-49, T-FR-50
FR_ExternalApproval	T-FR-51, T-FR-53, T-FR-54
FR_ApprovalPreflightValidate	T-FR-19, T-FR-44, T-FR-45, T-FR-46, T-FR-57B
FR_ApprovalRevision	T-FR-52, T-FR-55, T-FR-56
FR_DisableFeatures	T-FR-57, T-FR-57A, T-FR-57B
FR_ReferenceMaterial	T-FR-59, T-FR-60
FR_RequiredActionsList	T-FR-61, T-FR-62
FR_ActionRequiredEmail	T-FR-63
FR_ReadCourseInformation	T-FR-64, T-FR-65
FR_ExportCourseInformation	T-FR-66
FR_MacSSO	T-FR-73
FR_AuthorizationRBAC	T-FR-65, T-FR-73, T-FR-74, T-FR-75
FR_UpdateRoles	T-FR-75
FR_ExternalSupervisorVerification	T-FR-05, T-FR-06, T-FR-07
FR_ExternalSupervisorScope	T-FR-10, T-FR-42
FR_LogExternalErrors	T-FR-77
FR_RejectComponentDisable	T-FR-58
FR_DisplayTimestamps	T-FR-78
FR_DurableNotifications	T-FR-79

FR_ReverifyUsers	T-FR-67, T-FR-75, T-FR-76
FR_ImportEnrollment	T-FR-67, T-FR-68
FR_UpdateEnrollment	T-FR-70
FR_EnrollmentDiscrepency	T-FR-69
FR_NeedsGroup	T-FR-71
FR_AssignTA	T-FR-72

Table 3: Traceability Between Functional Requirements and System Tests

Non-Functional Requirement	Test Cases
NFR_ScreenSize	T-NFR-PORT-1
NFR_UsabilityFirstSuccess	T-NFR-USBLTY-1
NFR_LearnUnassisted	T-NFR-USBLTY-1
NFR_AccessibilityWCAG	T-NFR-ACC-1
NFR_PerfTail	T-NFR-PERF-1
NFR_Uptime	T-NFR-REL-1
NFR_Recoverable	T-NFR-REL-2, T-NFR-REL-3
NFR_PeakUsers	T-NFR-PERF-1
NFR_Browsers	T-NFR-PORT-1
NFR_Container	T-NFR-PORT-2
NFR_DeployUnassisted	T-NFR-INST-1
NFR_ChangeEffort	T-NFR-MAINT-2
NFR_LiveDisable	T-NFR-MAINT-1
NFR_Authenticate	T-NFR-SEC-1
NFR_Authorize	T-NFR-SEC-2
NFR_ConcurrencySafety	T-NFR-SEC-3
NFR_EssentialPersonal	T-NFR-SEC-9
NFR_EncryptTransit	T-NFR-SEC-4
NFR_EncryptRest	T-NFR-SEC-5
NFR_PasswordHash	T-NFR-SEC-6
NFR_AuditLog	T-NFR-SEC-7
NFR_AuditTamper	T-NFR-SEC-8
NFR_ComplianceAODA	T-NFR-ACC-1

Table 4: Traceability Between Nonfunctional Requirements and System Tests

5 Unit Test Description

5.1 Unit Testing Scope

5.2 Tests for Functional Requirements

5.2.1 Module 1

1. test-id1

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

2. test-id2

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

3. ...

5.2.2 Module 2

5.3 Tests for Nonfunctional Requirements

5.3.1 Module ?

1. test-id1

   Type:
   Initial State:
   Input/Condition:
   Output/Result:
   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

References

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Development plan. 2026a. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/DevelopmentPlan/DevelopmentPlan.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Hazard analysis. 2026b. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Module guide. 2026c. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/Design/SoftArchitecture/MG.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Module interface specification. 2026d. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Problem statement and goals. 2026e. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/ProblemStatement/ProblemStatement.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Software requirements specification. 2026f. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/SRS/SRS.pdf.

6 Appendix

6.1 Usability Test Proposal Values

The supervisor in **T-NFR-USBLTY-1** enters the values in Table 5. Values are kept short so that typing time does not affect the result. The fields are the required Project attributes in the SRS data dictionary.

Table 5: Proposal values for the supervisor task

Field	Value
Title	Course Planner
Course	Software Engineering
Background	Background
Objectives	Objectives
Impact	Impact
Completion status	None
Industry	Education
Project type	Non-hardware
Supervisor capacity	1
Confidentiality	None
IP	None

6.2 Usability Survey Questions

Participants complete this survey after attempting their assigned task.

Rating Questions
Participants rate each statement on a 1–5 scale, where 1 is **strongly disagree** and 5 is **strongly agree**.

1. I understood what I was expected to do.
2. I could find the information or action I needed.
3. The labels and instructions were clear.
4. The system behaved as I expected.
5. I could complete the task without help.
6. I would feel confident using this system for the same task.

Open-Response Questions
Participants answer each question in their own words.

7. Was there anything confusing? If yes, elaborate.
8. What, if anything, would you change to make the task easier?

6.3 SRS Validation Walkthrough and Checklist

This appendix contains the material used to validate the SRS (Patel et al., 2026f) with Dr. Smith (Section 3.2). It has three parts: the agenda of the walkthrough, the questions asked for each use case, and the checklist that Dr. Smith completes.

6.3.1 Agenda

The session lasts 45 minutes. A team member presents, and a second member records answers in the Supervisor Meeting issue. The checklist and agenda are given to Dr. Smith before the session.

Minutes	Topic	What we show
3	Introduction	Purpose of the session, how answers are recorded, how the checklist works
5	Status lifecycle	The project and team-project status lifecycle diagram and the slot (capacity) rule
8	`UC_ReviewProposal`	Main success scenario and the alternative flows (changes requested, rejected)
12	`UC_ApproveTeamProject`	Main success scenario, the automated checks, and the alternative flows
7	`UC_ImportEnrollment`	Main success scenario, the preview, and the alternative flows (Avenue unavailable, absent student)
5	`UC_FollowUpUnselected`	Main success scenario and the list of unselected projects
5	Remaining checklist items and close	Visibility rules, scope, and priorities

Table 6: Agenda of the SRS walkthrough with Dr. Smith

6.3.2 Questions for each use case

Status lifecycle.

- Are these the statuses that you would want to see for a project and for a team on a project, and are any missing or unnecessary?
- Is it right that a team's application does not use a slot, and that a slot is reserved only when the supervisor accepts?

`UC_ReviewProposal`**.**

- Is requesting changes and receiving a revised proposal enough, or do you need to edit a proposal yourself?
- Is it acceptable that an external supervisor is verified only when you approve a proposal that they submitted?

`UC_ApproveTeamProject`.

- Which parts of the review must never be automated?
- How should a change to an already approved project be handled in your course?
- In external approval mode, who records the outcome in the platform, and how soon?

`UC_ImportEnrollment`.

- Is the Avenue enrollment list the list that you use, and does it include each student's program?
- How do you handle students who join late or come from other departments today?
- What should happen to a student who is on a team but missing from a new import?

`UC_FollowUpUnselected`.

- Is "closed with no approved team" the right definition of an unselected project?
- What do you want to see on the list (for example project title, supervisor, and contact details)?
- Do you want the platform to send the thank-you message, or only produce the list?

6.3.3 Checklist

For each statement, the reviewer ticks *OK* if it is correct, *Change* if it needs to change (with a note)

Table 7: SRS validation checklist

#	Statement in the SRS	SRS reference	OK	Change
A. Project proposals				
1	A supervisor or student team submits a proposal. Only the instructor decides: list it (status Open), request changes, or reject it. TAs do not take part in proposal approval.	`UC_ReviewProposal`, `FR_ReviewProposal`, `SR_TAApproval`	☐	☐
2	Approving an unverified external supervisor's own proposal verifies them, and nothing else does. A verified supervisor can then see student profiles and student-proposed projects, but never another supervisor's projects.	`FR_ExternalSupervisorVerification`, `FR_ExternalSupervisorScope`	☐	☐
3	The required proposal fields are title, background, objectives, impact, completion status, industry, hardware or non-hardware, supervisor capacity, confidentiality, and IP. The optional fields are requirements, deliverables, proposed solution, and additional terms.	`FR_SupervisorProposal`, `FR_ProposalOptionalFields`	☐	☐
B. Selection and agreement				

4	An application does not use a slot. A slot is reserved when the supervisor accepts a team, a project is Full only when all its slots are reserved, and a team is Agreed only when both sides have accepted.	Capacity rule (Section on the lifecycle), `UC_SelectProject`	☐	☐
5	The project statuses (Draft to Closed) and the team-project statuses (Applied to Declined or withdrawn) are complete and correctly named.	Status lifecycle figure	☐	☐
6	A project is Closed when the selection period ends. A Closed project with no approved team is unselected, and the instructor can list these to follow up with their supervisors.	`UC_FollowUpUnselected`, `FR_ReadCourseInformation`	☐	☐
C. Team project approval				
7	Before a submission reaches the instructor, the platform checks that required fields and documents are present, the repository exists and is set up correctly, the instructor is a collaborator, every member is on exactly one team, the team has no other approved project, and the project is available.	`FR_ApprovalPreflightValidate`, `UC_ApproveTeamProject`	☐	☐
8	The instructor or a TA approves, rejects, or requests changes. The platform never decides automatically.	`SR_HumanApproval`, `SR_TAApproval`, `FR_ApprovalResponse`	☐	☐
9	A course can approve in the platform, record outcomes by hand from an external process, or (where an adapter exists) synchronize with one. Recording by hand is enough for the course's current process.	`FR_ExternalApproval`, approval mode definition	☐	☐
10	A change to an approved project stays pending, and the original approval stays in effect until the instructor accepts the change.	`FR_ApprovalRevision`	☐	☐
D. Enrollment, teams, visibility, and scope				
11	Enrollment is imported from Avenue (or a file) with a preview. Duplicates and malformed entries are flagged, students missing from a new import are flagged and never removed automatically, and late students can be added by hand.	`UC_ImportEnrollment`, `FR_ImportEnrollment`, `FR_EnrollmentDiscrepency`	☐	☐
12	Team size limits and permitted program combinations are set per course by the instructor. Teams that stop complying after a rule change are flagged for review, not removed. The values for the course are correct.	`FR_ValidateTeamRules`, course data in the data dictionary	☐	☐

13	The rules for who can read each type of record (proposals, projects, profiles, applications, approval feedback, and the TA's name) are correct, with nothing too open or too closed.	`DPR_VisibilityMatrix`, visibility tables	☐	☐
14	The items listed as not done (IP resolution, matching supervisors to student projects, meeting scheduling, direct messaging, post-approval feedback requests) are correct, and the priorities of the development phases match the instructor's biggest pain points.	Scope section, Development Plan	☐	☐

Other comments or corrections:

Reviewer: _____ Date: _____

6.4 Deployment Guide

This guide describes how to deploy Keystone on a Linux server with Docker installed. It is used in **T-NFR-INST-1**. It is a draft, and file and variable names will be updated to match the implementation. Replace values in angle brackets with those for your deployment.

1. Connect to the server:

   ```
   ssh <ssh-user>@<server-address>
   ```

2. Get Keystone, enter its directory, and check out the release tag provided with this guide (in the form `v<major>.<minor>.<patch>`):

   ```
   git clone https://github.com/Ayushpatel2026/Keystone.git
   cd Keystone
   git checkout <release-tag>
   ```

3. Create the configuration file from the template:

   ```
   cp .env.example .env
   ```

4. Open `.env` and set the following values:

   ```
   ADMIN_EMAIL=<admin-email>
   ADMIN_PASSWORD=<admin-password>
   TLS_CERT_PATH=<certificate-path>
   TLS_KEY_PATH=<key-path>
   ```

5. Build and start the system:

   ```
   docker compose up -d --build
   ```

6. Check that all containers are running:

   ```
   docker compose ps
   ```

7. Open `https://<server-address>` in a browser and sign in with the admin credentials.

6.5 Load Test Task Mix

The simulated users in **T-NFR-PERF-1** are split by role as in Table 8. Each time a simulated user starts a task, it picks one at random using the probabilities for its role in the table, which sum to 100% for each role. Each student team applies only to open projects it has not already applied to.

Table 8: Role split and task probabilities for the load test

Role	Users	Task	Probability (%)
Student	340	Browse and filter the project catalog	40
		View a project	30
		View the team's application status	15
		Reply to a project thread	10
		Apply to a project	5
Supervisor	40	View own projects	40
		View a project thread	30
		Reply to a project thread	20
		View applications to own projects	10
TA	12	View team compositions	60
		View a project	40
Course instructor	8	View proposals awaiting review	50
		View a proposal	30
		Submit feedback on a proposal	20

6.6 Likely Change Sample

Table 9 lists the changes used in **T-NFR-MAINT-2**. Each change is based on a likely change in the SRS (Patel et al., 2026f).

Table 9: Sample of likely changes for one week of maintenance

#	Change	Likely change
1	Add a project field that students can filter the catalog by	`LC_ProposalTemplates`
2	Add a team rule limiting the number of students from outside the course's department	`LC_TeamCourseRestrictions`
3	Add a TA review step before the instructor's final approval	`LC_ApprovalWorkflows`
4	Add a maintainer role that can only read the external-provider error log	`LC_Users`
5	Accept an enrollment file with a different column layout	`LC_Enrollment`

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

- Claude Sonnet 5.5: Medium
- GPT-6

7.2 How and Where Used

- Finding requirements not covered by currently written tests
  Sections Affected: FR System Tests, NFR System Tests
- Finding requirements covered by tests but the requirements weren't stated
  Sections Affected: FR System Tests, NFR System Tests
- Verifying that ideas were compatible and consistent with the document
  Sections Affected: FR System Tests, NFR System Tests
- Word choice, formatting, LaTeX syntax
  Sections Affected: General Information, SRS Validation Checklist, FR System Tests, NFR System Tests
- Extracting symbolic constants
  Sections Affected: NFR System Tests
- Reviewing content against the checklist and rubric for missing content and inconsistencies, providing suggested changes
  Sections Affected: Plan, NFR System Tests
- Brainstorming ideas within project scope, including automated testing technologies (e.g.: performance and mutation testing)
  Sections Affected: General Information, Automated Testing and Verification Tools
- Fixing a CI build issue
  Sections Affected: GitHub repository CI

7.3 Verification of AI-Generated Content

- Factual statements about previous documents (e.g.: requirement numbers and content) were cross-referenced with the relevant document (SRS, Hazard Analysis, etc.), and suggested technologies were checked against their official documentation
- The generated content was reviewed line by line and modified to align with the project's requirements and intended system behaviour

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Ayush:

1. What went well while writing this deliverable?

   I think it was pretty easy to come up with this plan. We all have extensive experience with verification and validation from our course work and industry experience: unit testing, integration testing, E2E testing, and CI/CD and automation pipelines. We also know how important it is to validate with customers that we are building the right thing, and I have experience with regular demoing to a customer for exactly that reason. All of our past experience has allowed us to come up with a good plan for this deliverable.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Time management and prioritization with so much going on right now. The plan itself came together easily, but finding the time to actually sit down and write it up was hard with midterms and other assignments all landing around the same time. I resolved this by blocking out a few set times in the week for this deliverable and writing it in smaller chunks, instead of waiting for a big free afternoon that never showed up.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   For me, it is unit testing in Python with pytest and in React with Jest. I have experience with Vitest in Angular, but I have never unit tested a React project, so I expect the ideas to carry over but the tooling to feel new. As a team, we will also need Playwright for the end-to-end tests, integration and concurrency testing against a real PostgreSQL database (including stubbing Avenue and the email service), and setting up Locust load tests and mutation testing in GitHub Actions.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   For React unit testing, two approaches are reading the Jest and React Testing Library documentation, and building a small throwaway React app with a few component tests to get

hands-on practice. I will do both: the documentation first for the basics, then the small project, because I learn fastest by doing things. YouTube tutorials are a good backup when the documentation does not make something click. The same two approaches (documentation plus a small practice project) work for the other skills on the team list, and the rest of the team will pick whichever suits them for the tool they own.

Waqar:

1. What went well while writing this deliverable?

   We had a lot of peer review items to do still during this deliverable, but we were able to manage them first and also work on the VnV plan. I think because we were able to manage our time and communicate effectively, we were able to get it all done. Additionally, we also tried to leave some buffer time this deliverable which was great meaning that we finished most of our work before the deadline in advance so that we had some time to review.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   The biggest pain point I experienced initially was trying to balance the peer review items with the VnV workload. However, after some time management, I allocated enough time to get through all the peer review tickets I need to while also having time for the VnV report. By allocating some time from my schedule into blocks ensured I had enough time in this cycle.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   Personally, I think the biggest learning curve for me will be integration testing as well as testing using playwright. I have done a good amount of work with python and unit testing on that front, but when it comes to testing more UI components through playwright I lack some experience. I plan to do some more research and learning on this specific topic to ensure I am able to provide high quality test cases for the project.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   The first way I plan to learn is to watch some youtube tutorials to ensure I have the basics downs and then also follow this up by making some practice test cases to make sure I know what I am doing. The second way is to ask some of my teammates who have experience using playwright as they can provide some more application sided help and context.

Andrew:

1. What went well while writing this deliverable?

   What went well while writing this deliverable was the level of detail that we ended up writing especially for our functional and non-functional requirement tests. It shows that our SRS was written well and it gave us a good idea about what type of workflows we needed to verify. We communicated very effectively to determine dependencies and ask each other questions to help with our own parts. With how this deliverable was written as well as the SRS, we have a good foundation for the remainder of the project.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   The biggest pain point I experienced with this deliverable was coming up with a comprehensive set of functional tests that would cover our complete set of functional requirements. The challenge came from the fact that we had many functional requirements and features associated with our project making it difficult to organize them into a complete set of tests without missing important functionality. The key to resolving this was going through each requirement one-by-one. Longer requirements that had many sub-requirements were split into different tests. This helped me become more organized and ensure complete coverage. It was also helpful in determining gaps in our SRS. My team members also helped point things out with their revisions.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   Personally, the skill I will have to contribute will be End-to-end tests with Playwright. I have been exposed to unit tests many times but have not had the opportunity to thoroughly perform End-to-end tests which is very exciting for me as I know how important it is to perform functional tests and simulate workflows and edge-cases. This will help us ensure that given the state of our application, the functionality in the SRS will be completely covered.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   I will continue to refine my skills in unit-testing via looking back at previous projects and courses especially SFWRENG 3S03 and 2AA4 where I wrote many unit tests and learned about various important techniques. To supplement this, I will also need to do lots of research especially to learn about how Playwright works and how it is used in e2e tests. After the research and watching tutorials, I will need to practice it myself before fully using it. Every member discusses their approach to how they will learn the skills we need and Table 1 in this document outlines who is the owner of each type of test.

Ahmed:

1. What went well while writing this deliverable?

   The number of the NFR system tests and their purpose naturally followed from the SRS. Additionally, writing tests helped with refining NFRs, since good NFRs are ones that can be tested. For example, writing `T-NFR-SEC-6: Password Storage` helped expose a gap between the SRS and the mental version of the requirement involving the salting of passwords before hashing, which was mentally noted and ended up implicitly described (something along the lines of "same password used by multiple users has a different hash") but not described in the SRS.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Usability testing was the most involved and difficult part of the NFR system tests, as it aims to quantify a system quality which is often thought of as a general feel. Writing the SRS partially solved this, since usability requirements approximate the usability of the system

through heuristics like first-time use task completion and the ability to use the platform without assistance. However, it was unclear whether this usability test was enough to rule out major issues.

This was resolved by ensuring tests have scenarios geared towards the stakeholder being tested, and that the scenario leaned towards being higher in complexity. If the test passes with the more complex scenarios, we can be confident that the simpler ones do not have major usability issues.

Additionally, browser tests were geared towards usability. Control buttons are tested to be visible, interactable, and sufficiently large. This way, rather than just testing whether the application can run on each browser, the tests can also indirectly provide some redundancy with usability.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   I will need to understand exactly how to use Playwright and enable browser testing, ensure accessibility tests are following the required standard that needs to be complied with.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   One approach would be reading the documentation created for Playwright and reading relevant sections until an understanding is reached. Another approach would be to ask an AI model questions specific to what this project will test or even feed it the VnV tests and ask it for guidance.

   The approach I will go with is to read the documentation's relevant sections, but also skim all other major sections which may not be applicable to this project. The reason is simple: Industry standard tools often receive regular updates expanding features and deprecating old ones. AI models often lean into old ways since they train on existing code which is out of date by the time the models are published. Additionally, AI models will simply try to fulfil the task you provide and nothing more. This can be extremely limiting and I've observed this in my experience using AI models to implement new libraries at my job. While they can troubleshoot, provided enough context, they simply lack the ability to follow rapidly changing best practices. Documentation for these tools can often spark new ideas of better ways to use them than were thought of during planning. This is also the reason for skimming "irrelevant" sections.